



MA-223

Semester Project

Group 42

FINDING PARAMETERS OF A DC MOTOR WITH STATISTICS

Employing statistical analysis of physical measurements in order to estimate parameters of a DC motor system model.

EIRIK KALDHOL STRANDMAN(784), Inge Johan Straumsøy Johansson(632)

SUPERVISOR(S)

Svein Olav Nyberg

Contents

Sammendrag	i
1 Introduction	1
1.1 Abstract	1
1.2 Goals	1
1.3 Motivation	1
2 Theory	2
2.1 DC Motor Equations	2
2.1.1 Locked rotor test	3
2.2 No-Load Test	3
2.2.1 Motor constant $[K_m]$ and $[E]$	3
2.3 Components	4
2.3.1 Power Sensor - INA219	4
2.3.2 Absolute Position Encoder - AS5600	5
2.3.3 Microcontroller - Arduino Mega	5
3 Methods	6
3.1 Design	6
3.2 Data Processing	8
3.2.1 Back-EMF and internal resistance	8
3.2.2 Angular Velocity	10
3.3 Programming	12
4 Results	13
4.1 Linear Regression	13
4.2 Hyperparameter	14
4.2.1 Prior	14
4.2.2 Posterior	14
4.3 Posterior Distributions	14
4.4 Credibility and Predictive interval at 99% confidence for $y(\mathbf{x})$ and $Y_+(x)$	15
4.4.1 Credibility and Predictive interval at 800 RPM with 99% confidence for $y(\mathbf{x})$ and $Y_+(x)$	15
5 Discussion and conclusion	16
5.1 Discussion	16
5.1.1 Angular Velocity Measurements	16
5.1.2 Applied Voltage Measurements	16
5.1.3 Time Stamping Measurements	16
5.1.4 Linear Regression	16
5.1.5 Further Improvements	17
5.2 Conclusion	17

A Programming	18
A.1 C++	18
A.1.1 Arduino Motor Logging script	18
A.2 Python	20
Bibliografi	48

Chapter 1

Introduction



Figure 1.1: Test setup

1.1 Abstract

The report will go through the process of estimating the parameters of a steady-state brushed DC motor model using measurements from affordable electronic components. This is done using statistical analysis like linear regression. Inference is then applied to the estimate(s).

1.2 Goals

This project aims to conduct physical measurements on brushed DC motors using affordable, easy-to-use electronic components and construct a steady-state model through statistical analysis, to validate the inference is applied to the estimates.

1.3 Motivation

The project combines mechatronics, control theory, and last but not least statistics to improve our understanding and provide practical experience on the subjects.

The choice of the brushed DC motor stems from its widespread use in engineering and frequent application within our university. By modeling this motor, we intend to facilitate its utilization in future projects such as servo control, ball-on-beam regulation, and inverted pendulums.

Chapter 2

Theory

2.1 DC Motor Equations

A DC motor's behavior can be modeled using the two following differential equations:

Electrical equation:

$$V = i_a R_a + \dot{\theta} k_m + \frac{di_a}{dt} L \quad (2.1)$$

- V - Total voltage drop of the motor in volts $[V]$.
- i_a - Total current through the motor in ampere $[A]$.
- R_a - Total resistance in the motor in ohms $[\Omega]$.
- $\dot{\theta}$ - Angular velocity of the motor in radian per second $[\frac{rad}{s}]$.
- k_m - Motor constant.
- $\frac{di_a}{dt}$ - Angular velocity of the motor in radian per second $[rad/s]$.
- L - Motor inductance in henries $[H]$.

Mechanical equation:

$$I\ddot{\theta} = i_a k_m + B\dot{\theta} + T_s \quad (2.2)$$

- $T = I\ddot{\theta}$ - Motor torque in newton meters $[Nm]$.
- I - Polar moment of inertia of the rotor $[m^4]$.
- $\ddot{\theta}$ - Angular acceleration of the rotor $[\frac{rad}{s^2}]$.
- B - Viscous loss coefficient.
- T_s - Static friction losses in the motor in newton meters $[Nm]$.

For our purposes, we will only look at the steady-state electrical model, simplifying the equations to:

$$V = i_a R_a + \dot{\theta} k_m \quad (2.3)$$

To find the constants R_a and k_m , two tests are performed:

- Locked rotor test
- No load test

2.1.1 Locked rotor test

In a locked rotor test, the rotor is locked in place. This way, the rotation speed $\dot{\theta} = 0$. This simplifies the differential equations to

Electrical equation:

$$V = i_a R_a + \frac{di}{dt} L$$

Mechanical equation:

$$I\ddot{\theta} = iK_m + T_s$$

In order to determine the internal resistance of the motor, commonly denoted as armature resistance R , a constant current test can be performed. This results in $\frac{di}{dt}L = 0$, with the following equation:

$$V = i \cdot R$$

Now it is possible to solve for R :

$$R_a = \frac{V}{I} \quad (2.4)$$

2.2 No-Load Test

In the no-load test, we allow the rotor to spin freely. By measuring the applied voltage V , current i , and rotor position θ we can determine the rest of the parameters.

2.2.1 Motor constant $[K_m]$ and $[E]$

The motor constant can be found using linear regression. We can see this by comparing the electrical equation with a constant current and the line function:

$$V(\dot{\theta}) = i_a \cdot R_a + K_m \cdot \dot{\theta} \quad (2.5)$$

$$y(x) = a + bx \quad (2.6)$$

$$V(\dot{\theta}) = y(x), \quad i_a \cdot R_a = a, \quad K_m \cdot \dot{\theta} = bx$$

However, we encounter an issue with $a = i_a \cdot R_a$, since a is constant, while $i_a \cdot R_a$ is not. We can remedy this by rearranging the equation and setting $a = 0$:

$$(V - iR) = K_m \dot{\theta} \quad (2.7)$$

$$y = (V - iR) = E$$

Now the back-EMF $[E]$ is equal to y . The back-EMF constant is also equal to the motor constant when using metric units. Keep in mind, in this report $k_v = \frac{V}{RPM} = E$ will be used, as RPM is an easier unit to understand outside of engineering.

2.3 Components

2.3.1 Power Sensor - INA219

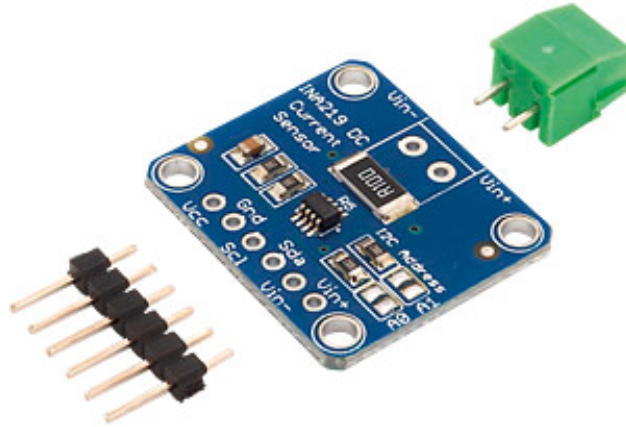


Figure 2.1: INA219 breakout board [7]

The Texas Instruments INA219 is a shunt resistor current sensor. It measures the current through the load, in this case the motor, by measuring the voltage drop across a precision resistor in series with the load. Additionally, it can also measure the voltage over the load or applied voltage. It has a built in Analog to Digital Converter (ADC), which allows a quick low noise measurement to be sent over serial by I2C [6].

2.3.2 Absolute Position Encoder - AS5600

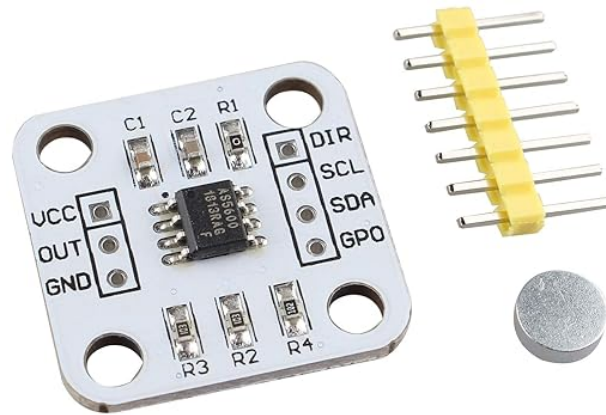


Figure 2.2: AS5600 breakout board with magnet. [5]

The AMS AS5600 is an absolute position encoder. It can precisely measure the position of the shaft with a resolution of 4096 bit. It uses the hall-effect to measure the magnetic field of a bipolar magnet mounted to the shaft. The position is then transmitted over serial via I2C [1].

2.3.3 Microcontroller - Arduino Mega

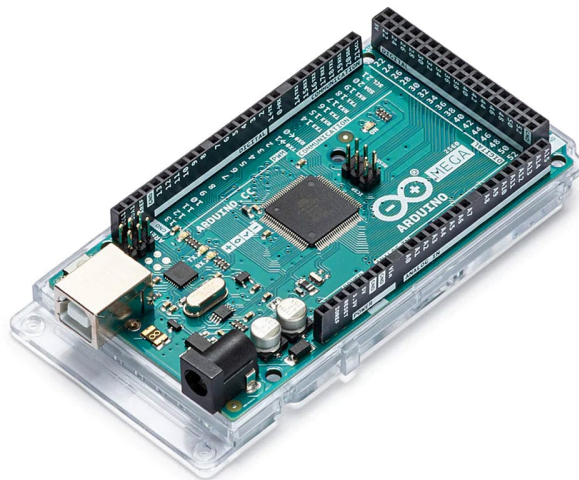


Figure 2.3: AS5600 breakout board with magnet. [2]

The Arduino Mega is popular microcontroller with universities, learning institutions and hobbyist. It is built on the Arduino framework, making it simple and familiar to program for a beginner [2].

Chapter 3

Methods

3.1 Design

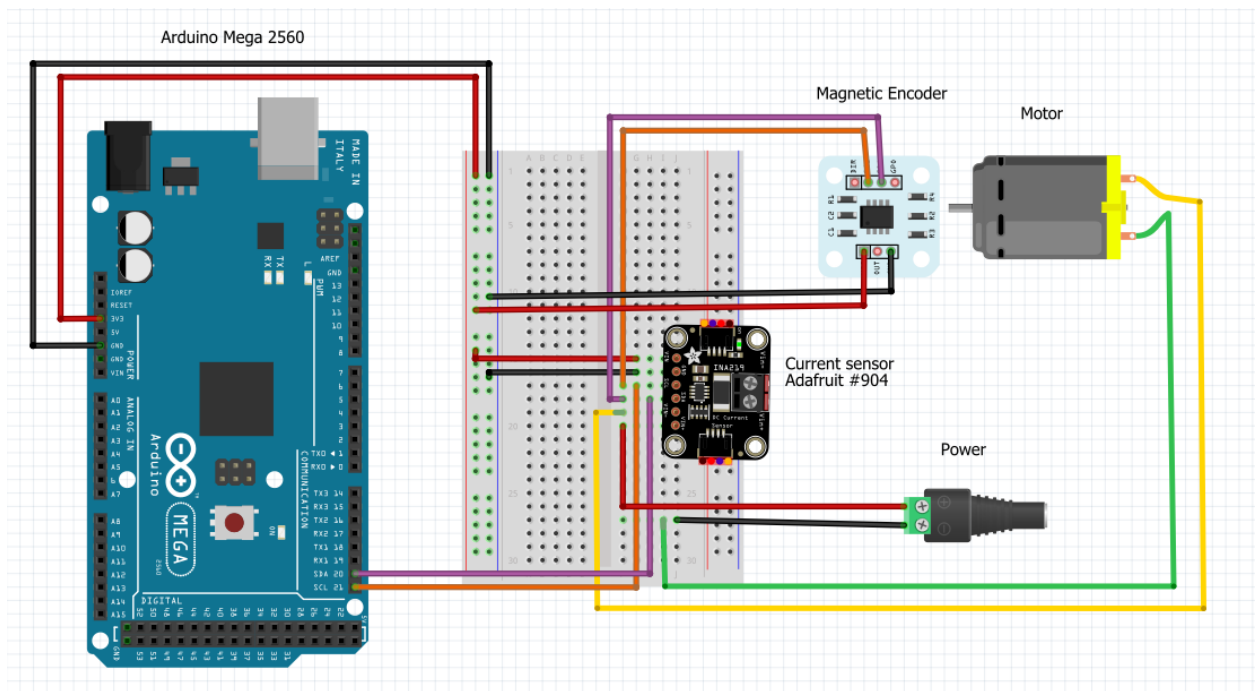


Figure 3.1: Wiring diagram

Figure 3.1 shows the simplified wiring diagram for the set-up.

This setup includes an Arduino Mega for data logging, an INA219 current sensor, and AS5600 absolute encoder. The motor tested is a Diligent DC brushed motor equipped with a 1:19 gearbox [3], as it is commonly used by the university.

Figure 3.2 on the following page illustrates the test setup. It is equipped with a power supply adjustable in 0.1V increments, allowing for precise measurements at 0.5V intervals [4].



Figure 3.2: Test setup

3.2 Data Processing

The data collected from the motor includes the following:

- Angular position [Deg]
- Bus voltage [V]
- Voltage drop over the current shunt resistor [mV]
- Current draw [mA]
- Timestamps [s]

Before the measurements can be used, the raw data requires some processing and filtering.

3.2.1 Back-EMF and internal resistance

By using equation 2.7, we can calculate the back-EMF generated by the motor. Since we know the current and applied voltage, we have to determine the internal resistance of the motor. For this, we use equation 2.4, but we modify it slightly to include the voltage drop from the current shunt resistor.

$$R = \frac{V_{Bus} - V_{shunt}}{I} \quad (3.1)$$

By applying this formula to the dataset while the rotor is stationary, and averaging the result, we get $R = 5.96\Omega$

We then insert this value into equation 2.7 and calculate the B-EMF for all currents measured. The before and after results can be seen in figure 3.3. We have now properly accounted for the voltage drop caused by the internal resistance of the motor.

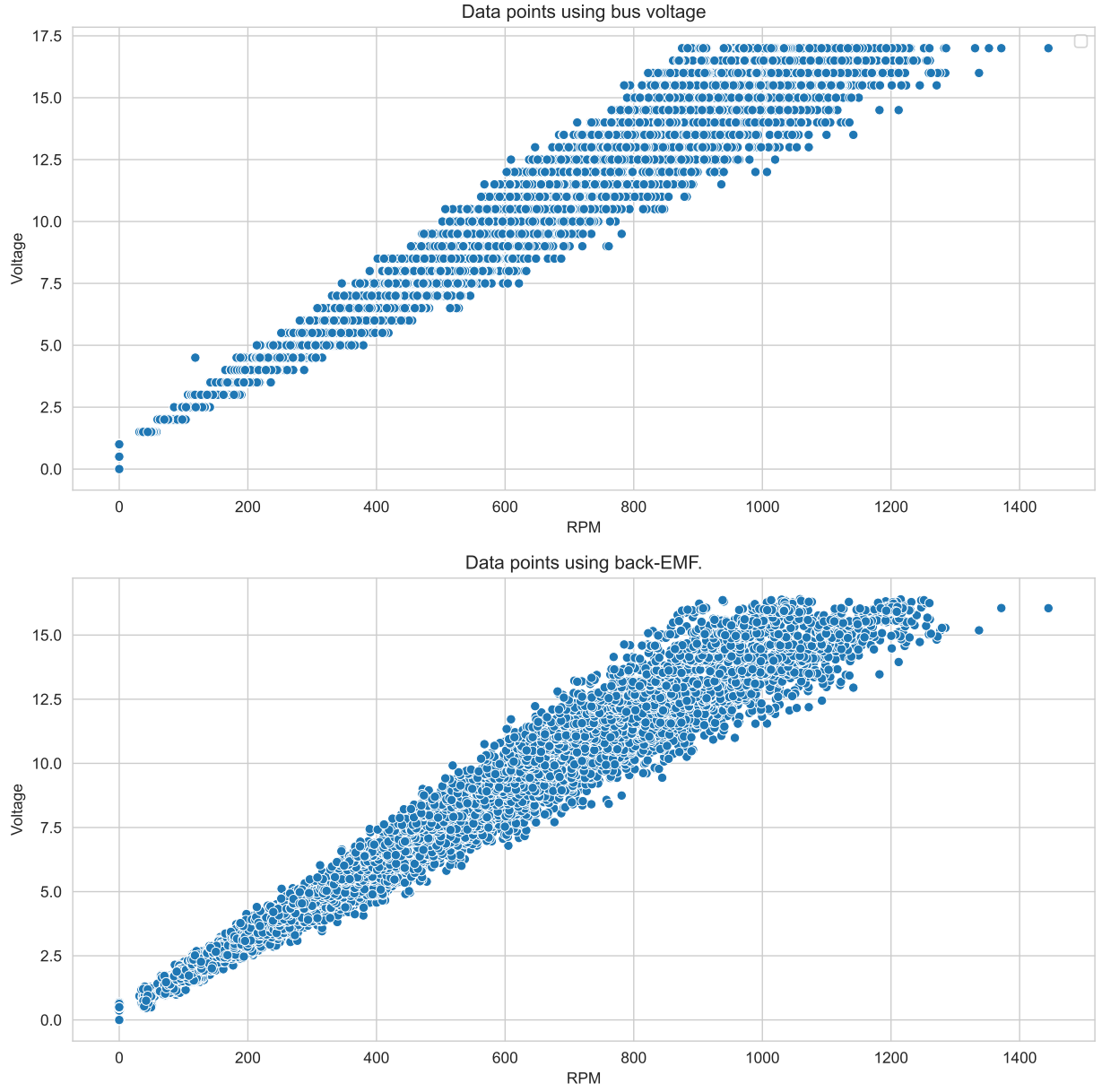


Figure 3.3: Comparison between using bus voltage and estimated B-EMF

3.2.2 Angular Velocity

In order to determine the angular velocity of the motor's rotor, the derivative of the rotor's position was taken with respect to time, using the following equation.

$$V_r = \dot{\theta} = \frac{d}{dt}(\theta) \cdot \frac{60}{360} \quad (3.2)$$

Resulting in the following graph:

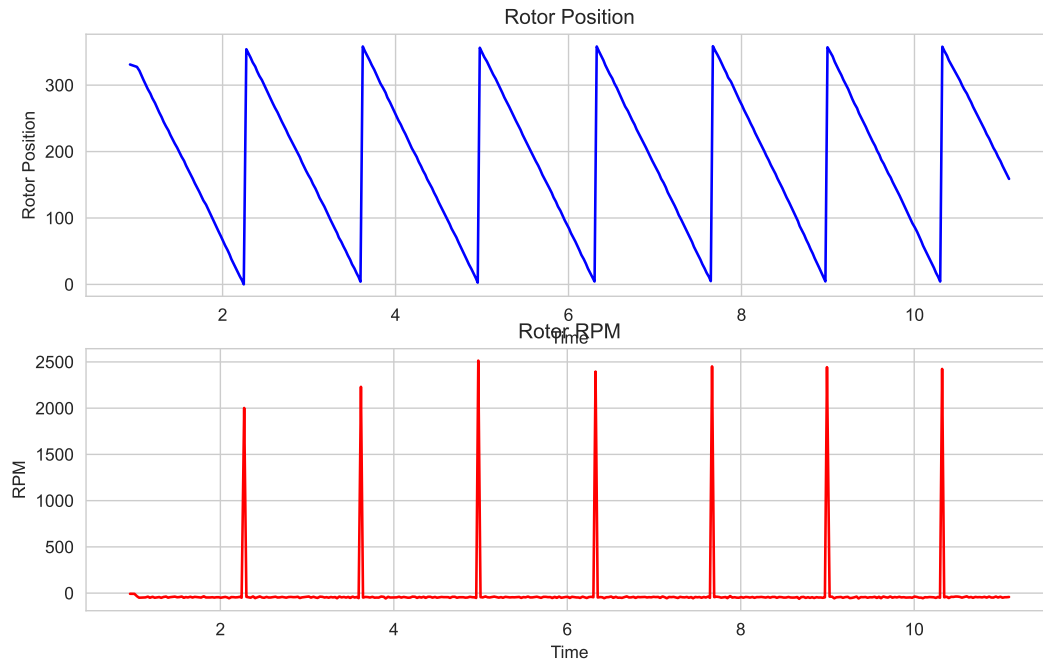


Figure 3.4: Raw position data and velocity derivative.

From figure 3.4 above, three issues are encountered with the data:

- The rotor position goes from 0-359 degrees, resetting every 360 degrees.
- Every time the rotor position resets, the estimated velocity goes to the moon.
- The motor spins Counter-Clock-Wise (CCW).

To improve the angular velocity estimate, we can do two things:

- Invert the rotor position data, from 359-0 to 0-359 degrees.
- Remove all negative values from the angular velocity estimate since we know the motor spins in one direction at a constant speed.

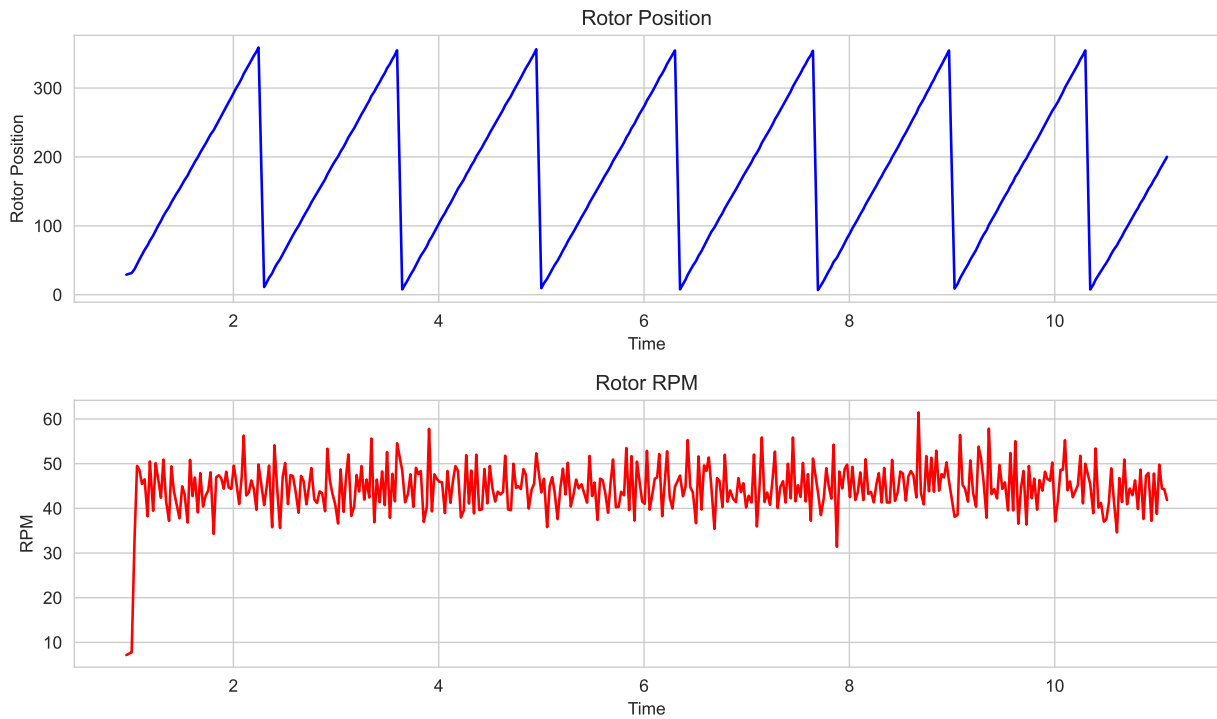


Figure 3.5: CW Motor position and negative velocity values removed.

Now the data is looking a lot more reasonable. However, the angular velocity data still contains some artifacts. We can filter this out with outlier detection, and then interpolate the missing values.

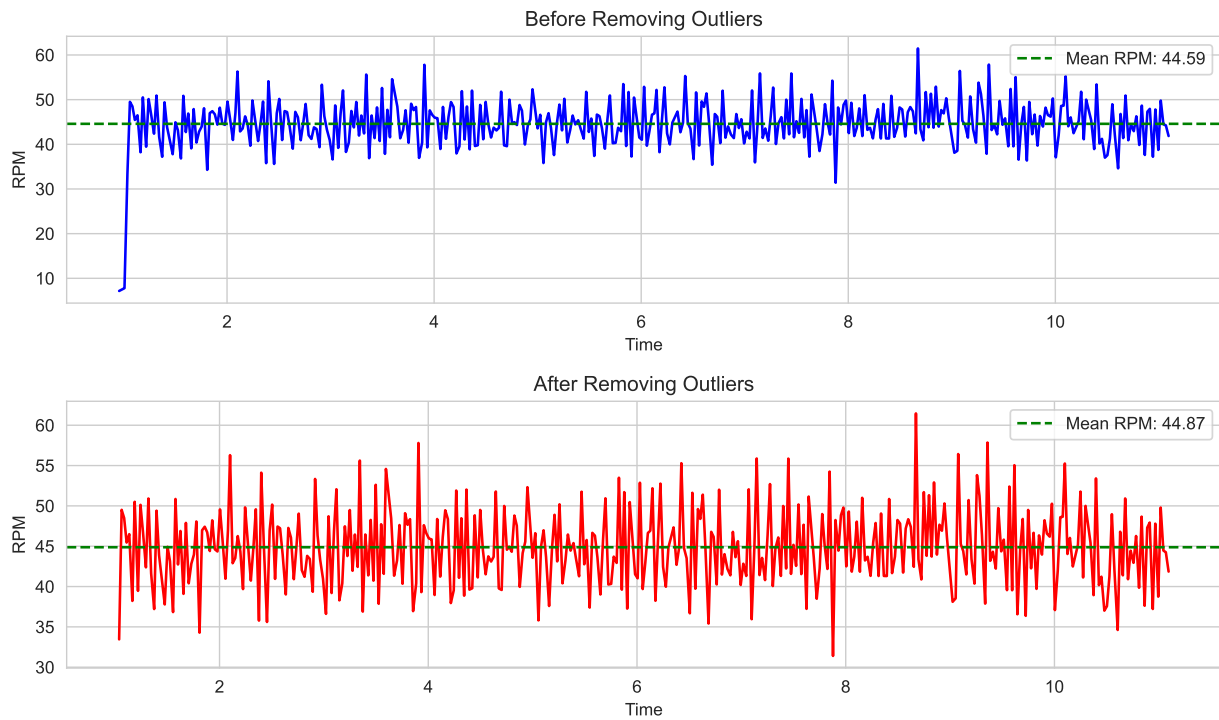


Figure 3.6: CW Motor position and negative velocity values removed.

With this data filtering seen in figure 3.6, we will assume that we have a decent angular velocity data set, and can apply this to the entirety of the data.

3.3 Programming

For the method of programming, see the Appendix in the "Programming" section. it contains Jupyter Notebooks which explain the process.

Chapter 4

Results

4.1 Linear Regression

$$y(x) = 6.90 + 0.014x \quad (4.1)$$

$$\vec{\beta} = \begin{bmatrix} \alpha_* \\ \beta \end{bmatrix} = \begin{bmatrix} 6.9176 \\ 0.0140 \end{bmatrix}$$

$$SS_e = 7099.9009$$

$$S_e^2 = 0.6689$$

$$S_e = 0.8179$$

$$R^2 = 0.97$$

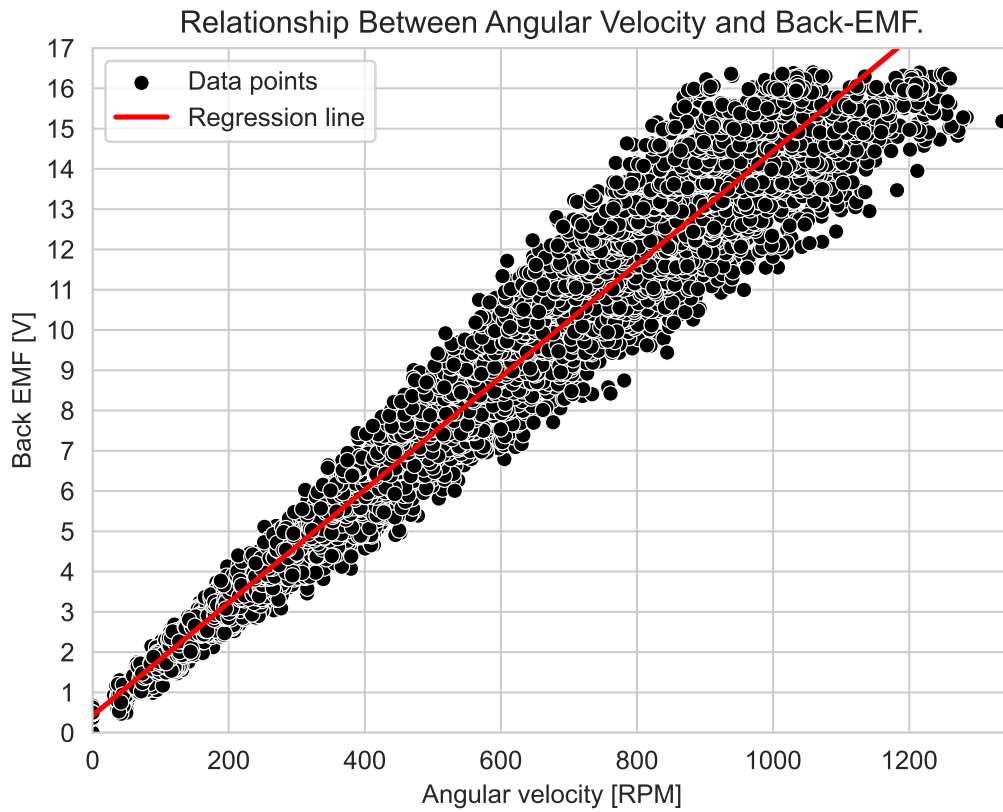


Figure 4.1: Measured RPM vs Estimated BEMF.

4.2 Hyperparameter

4.2.1 Prior

P0|=

- $v_0 = -2$
- $S_0 = 0$

4.2.2 Posterior

P1|=

- $\bar{x} = 463.13$
- $\vec{\beta} = \begin{bmatrix} \alpha_* \\ \beta \end{bmatrix} = \begin{bmatrix} 6.90 \\ 0.014 \end{bmatrix}$
- $v_1 = 10614$
- $S_1 = 7093.64$

4.3 Posterior Distributions

- $\tau \sim \gamma(5307.00, 3546.8218)$
- $\alpha_* \sim t_{(6.90, 7.9344 \cdot 10^3, 10614)}$
- $\beta \sim t_{(0.014, 2.3605 \cdot 10^5, 10614)}$
- $y(x) \sim t_{(6.90+0.014(x-463.13), 0.8175 \cdot \sqrt{7.9344 \cdot 10^3 + 2.3605 \cdot 10^5 \cdot (x-463.13)^2}, 10614)}$
- $Y_+(x) \sim t_{(6.90+0.014(x-463.13), 0.8175 \cdot \sqrt{1+7.9344 \cdot 10^3 + 2.3605 \cdot 10^5 \cdot (x-463.13)^2}, 10614)}$

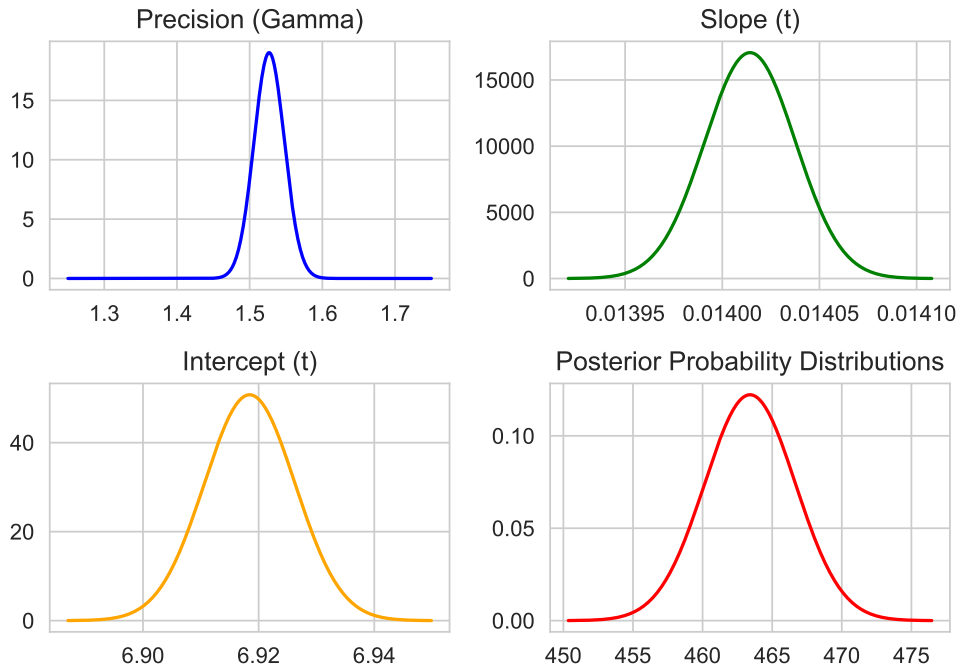


Figure 4.2: PDFs for different values.

4.4 Credibility and Predictive interval at 99% confidence for $y(x)$ and $Y_+(x)$.

- $I_{0.01}(x) = 6.90 + 0.014(x - 463.13) \pm 1.9021 \cdot \sqrt{7.9344 \cdot 10^3 + 2.3605 \cdot 10^5 \cdot (x - 463.13)^2}$
- $I_{0.01}^+(x) = 6.90 + 0.014(x - 463.13) \pm 1.9021 \cdot \sqrt{1 + 7.9344 \cdot 10^3 + 2.3605 \cdot 10^5 \cdot (x - 463.13)^2}$

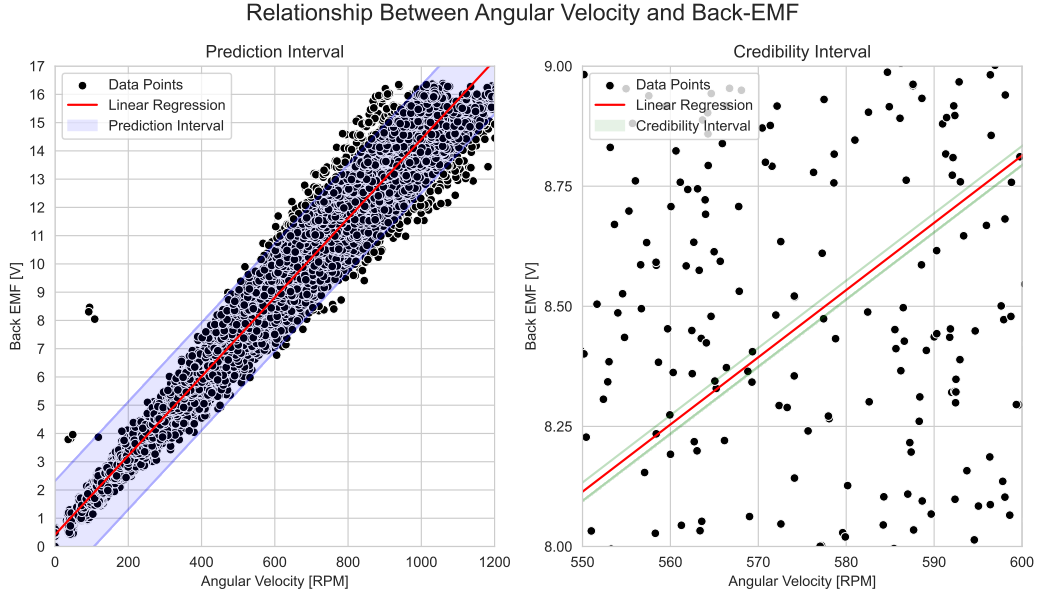


Figure 4.3: Credibility and prediction interval at 99% confidence. Note right plot is scaled to show credibility interval.

4.4.1 Credibility and Predictive interval at 800 RPM with 99% confidence for $y(x)$ and $Y_+(x)$.

- $I_{0.01}(800RPM) = (11.5861V, 11.6383V)$
- $I_{0.01}^+(800RPM) = (9.7099V, 13.5145V)$

Chapter 5

Discussion and conclusion

5.1 Discussion

5.1.1 Angular Velocity Measurements

As can be seen in the angular velocity section for data processing in methods, the absolute encoder was not well suited for velocity measurements, requiring heavy post-processing to provide good results. Using an incremental encoder might simplify the data processing and reduce the cost of a steady-state test.

5.1.2 Applied Voltage Measurements

The INA219 was unable to provide usable applied voltage estimates, instead, an estimate was used based on the voltage programmed on the lab power supply for the sample. While this does not have a large impact on the results, it could be useful to have a voltage measurement in the future to automate the process.

5.1.3 Time Stamping Measurements

During the measuring process, the microcontroller takes in the measurements and dumps them on the serial bus, the PC then reads these measurements and uses its internal clock to add a timestamp. While this did save cost, a future improvement would be to implement a Real Time Clock (RTC) on the microcontroller and store the measurement locally during the test, then upload it the the PC on completion. This would avoid most of the time delay between the measurement and time stamps during data transmission, and reduce the bottleneck in the data transmission.

5.1.4 Linear Regression

The linear regression managed to provide an equivalent line function very close to the data set, with a r^2 value of 0.97, indicating a good fit. During testing, it was noticed that we could take the intercept α and subtract it from the estimated back-EMF E data set to reduce the intercept to zero. This could improve the fit but was left out for education purposes. It might also be explained by the static friction T_s as noted in equation 2.2.

5.1.5 Further Improvements

- Replace the absolute position encoder with an incremental encoder.
- Measure the applied voltage directly.
- Add real time clock (RTC) to the Arduino to get accurate time stamping.
- Log data to SD card, and then send the entire batch over to the PC to avoid bottlenecking at the serial print.

5.2 Conclusion

In conclusion, we designed and built a test set-up and performed measurements on a brushed DC motor.

Performing a locked-rotor test to estimate the internal resistance of the motor, then performing a no-load test to determine the back-EMF constant $[E]$, using statistical linear regression.

We then performed inference on the model to determine the accuracy.

With a $r^2 = 0.97$ and predictive and credibility intervals seen in figure 4.3, the model appears accurate.

Appendix A

Programming



Scan the QR code above to access the Github Repository or use the link below:
<https://github.com/Eirik2020/Estimating-Parameters-of-a-DC-Motor-Using-Statistics>

NOTE: Repository is set to private until after the exam period.

A.1 C++

A.1.1 Arduino Motor Logging script

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_INA219.h>

#define AS5600_ADDRESS 0x36

Adafruit_INA219 ina219;

void setup() {
  Wire.begin();
  Serial.begin(9600);

  if (!ina219.begin()) {
    Serial.println("Failed to find INA219 chip");
    while (1);
  }

  ina219.setCalibration_32V_2A();
}
```

```

void loop() {
    // Read and print AS5600 data
    Wire.beginTransaction(AS5600_ADDRESS);
    Wire.write(0x0E);
    Wire.endTransmission();
    Wire.requestFrom(AS5600_ADDRESS, 2);

    if (Wire.available() >= 2){
        int highByte = Wire.read();
        int lowByte = Wire.read();
        int angle_raw = (highByte << 8) | lowByte;

        float angle_degrees = (angle_raw * 360.0) / 4096.0;
        Serial.print(angle_degrees);
        Serial.print(", ");
    }

    // Read and print INA219 data
    float shuntVoltage_mV = ina219.getShuntVoltage_mV();
    float busVoltage_V = ina219.getBusVoltage_V();
    float current_mA = ina219.getCurrent_mA();
    Serial.print(busVoltage_V);
    Serial.print(", ");
    Serial.print(shuntVoltage_mV);
    Serial.print(", ");
    Serial.println(current_mA);
}
}

```

A.2 Python

Motor Data Logging From Serial Bus

Python Packages

```
In [ ]: import serial      # Used for serial communication over USB.
import time      # Used for timekeeping.
import os        # Used to check and access files.
import pandas as pd # Used to generate data structures.
```

Configuration

- port: The port refers to the COM port or USB port the microcontroller is connected to.
- baud_rate: Set baud rate for communication, match this with the microcontroller code.
- csv_file: The name of the csv file for the generated data.
- CSV_folder_name: The name of the folder the csv shall be stored in.
- n_samples: The number of samples you wish to collect.

```
In [ ]: port = 'COM3'
baud_rate = 9600
csv_file = 'test.csv'
CSV_folder_name = "CSV_files"
n_samples = 100
```

Start recording

```
In [ ]: # Open the serial port
ser = serial.Serial(port, baud_rate, timeout=1)
print("Connection established.")

# Initialize empty lists to store the data
bus_voltage = []
shunt_voltage = []
current = []
angle = []
t = []

# Start time
start_time = time.time()
print("Starting recording...")

# Collect data
for i in range(n_samples):
    line = ser.readline().decode('utf-8').strip()
    if line:
        # Parse and store data
        data = line.split(',')
        if len(data) == 4:
            angle_raw = float(data[0])
            bus_voltage.append(float(data[1])) # Convert to float
            shunt_voltage.append(float(data[2])) # Convert to float
```

```

        current.append(float(data[3])) # Convert to float
        angle.append(angle_raw)
        t.append(time.time() - start_time)

# Close the serial port
ser.close()

# Calculate total time spent logging
time_tot = time.time() - start_time

# Report
print("Recording Complete.")
print("Time spent logging: {:.2f} seconds".format(time_tot))
print("Storing data...")

```

Connection established.
 Starting recording...
 Recording Complete.
 Time spent logging: 3.36 seconds
 Storing data...

Save data and report

```

In [ ]: # Create a DataFrame from the collected data
df = pd.DataFrame({
    'Angle': angle,
    'V_bus': bus_voltage,
    'V_shunt': shunt_voltage,
    'current': current,
    'time': t
})

# Check if the file exists, and append or create accordingly
if os.path.isfile(csv_file): # File exists
    # Read the CSV file into a DataFrame
    df_old = pd.read_csv(csv_file)

    # Concatenate the old DataFrame with the new one
    df = pd.concat([df_old, df], axis=1)

# Save data
file_path = os.path.join(CSV_folder_name, csv_file) # Generate file path
df.to_csv(file_path, index=False) # Save df

# Report
print("-----")
print("Data saved to {}: {} samples recorded in {:.2f} seconds".format(csv_file, n,
print("-----")

```

 Data saved to test.csv: 100 samples recorded in 3.36 seconds

DC Motor, Locked Rotor Test, data processing

Packages

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
import seaborn as sns
import os
import json
```

Configuration data.

```
In [ ]: # Folder name
CSV_folder_name = "CSV_files"
figure_folder_name = "figures"
json_folder_name = "json"

# Motor data file name and default values
motor_file = 'motor_data.json'
default_motor_info = {
    'R_a': 1,
    'K_V': 1,
    'K_m': 1
}

# Option to save data and figures ( True = Save data and figures, False = Don't save)
SAVE = False
```

Load data set

```
In [ ]: # Load data from CSV file
file_name = "locked_rotor2.csv" # CSV name
file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
raw_df = pd.read_csv(file_path) # Read CSV file.
```

Calculating the internal resistance of the motor.

Equation for calculating internal resistance:

$$R_a = \frac{V - V_{shunt}}{i_a} \quad (1)$$

Where:

- R_a = Internal resistance of the motor in ohms. $[\Omega]$.
- V = Applied voltage in volts $[V]$.
- V_{shunt} = Voltage drop from current measuring resistor in volts $[V]$.

- i_a = Current passing through motor in ampere [A].

```
In [ ]: # Select example sample from data set
sample = 5
voltage_step = 1 # Voltage step-up per sample, 1 = 1V, 2V, 3V , 4V etc...

# Create new dataframe for rotor data, containing rotor position and time
voltage_data = pd.concat([raw_df.iloc[:, (5*sample + 1)], raw_df.iloc[:, (5*sample
voltage_data.columns = ['V_bus', 'V_shunt', "Current", "Time"]

# Change V_shunt from mV to V and Current from mA to A
voltage_data['V_shunt'] = voltage_data['V_shunt'] / 1000
voltage_data["Current"] = voltage_data["Current"] / 1000

# Calculate armature resistance
voltage_data["Armature Resistance"] = ( voltage_step*sample - voltage_data['V_shunt
armature_resistance = abs(voltage_data['Armature Resistance'].mean()) # Calcualte n

# Print estimated resistance
print("Estimated armature resistance for sample {:.0f}, with voltage step of {:.0f})
```

Estimated armature resistance for sample 5, with voltage step of 1V: 6.13Ω.

Apply above calculations to the entire data set

```
In [ ]: # Voltage step-up per sample, 1 = 1V, 2V, 3V , 4V etc...
voltage_step = 1

# Create an empty DataFrame for motor constant
resistance_array = np.array([])

# Iterating through every fifth column
for sample in range(0, len(raw_df.columns), 5):
    ## Create new dataframe for voltage data
    voltage_data = pd.concat([raw_df.iloc[:, (sample + 1)], raw_df.iloc[:, (sample
    voltage_data.columns = ['V_bus', 'V_shunt', "Current", "Time"]

    ## Change V_shunt from mV to V and Current from mA to A
    voltage_data['V_shunt'] = voltage_data['V_shunt'] / 1000
    voltage_data["Current"] = voltage_data["Current"] / 1000

    ## Calculate applied voltage for dataset
    voltage_data["V_applied"] = voltage_step*(sample/5)

    ## Calculate armature resistance.
    voltage_data["Armature Resistance"] = ( voltage_data["V_applied"] - voltage_dat

    ## Replace inf with NaN
    voltage_data.replace([np.inf, -np.inf], np.nan, inplace=True)

    ## Remove rows with NaN values
    voltage_data = voltage_data.dropna()

    ## Store resistance estimates in array
    resistance_array = np.append(resistance_array, abs(voltage_data['Armature Resis
```

```
# Calculate mean armature resistance
armature_resistance = np.mean(resistance_array)

# Calculate mean, mode and median
armature_resistance = np.mean(resistance_array)

# Print values
print("Estimated mean armature resistance: {:.2f}Ω.".format(armature_resistance))
print("")

# Save data
if SAVE:
    ## Generate file path
    file_path = os.path.join(json_folder_name, motor_file)

    ## Check if the file exists
    if os.path.exists(file_path):
        ### If file exists, read the values from it
        with open(file_path, 'r') as json_file:
            motor_data = json.load(json_file)
    else:
        ### If file doesn't exist, use default values
        motor_data = default_motor_info

    ## Print old motor data
    print("Old motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")

    ## Save resistance
    motor_data['R_a'] = armature_resistance

    ## Save the updated dictionary to the JSON file
    with open(file_path, 'w') as json_file:
        json.dump(motor_data, json_file)

    ## Print new motor data
    print("New motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")
```

Estimated mean armature resistance: 5.80Ω.

DC Motor, No Load Test, data processing

Packages

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
import seaborn as sns
import os
import json
import warnings
```

Configuration

```
In [ ]: # Folder name
CSV_folder_name = "CSV_files"
figure_folder_name = "figures"
json_folder_name = "json"

# Set Seaborn style
sns.set_style("whitegrid")

# Option to save data and figures ( True = Save data and figures, False = Don't save)
SAVE = False

# Ignore warnings
warnings.filterwarnings("ignore", message="invalid value encountered in subtract")
```

Load Data Set

```
In [ ]: # Load data from CSV file
file_name = "motor2.csv" # CSV name
file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
raw_df = pd.read_csv(file_path) # Read CSV file.
voltage_step = 0.5 # Voltage step, 1 = 1V, 2V, 3V, 4V etc...

# Load data from json file
motor_file = "motor_data.json" # CSV name
file_path = os.path.join(json_folder_name, motor_file) # Generate file path
with open(file_path, 'r') as json_file:
    motor_data = json.load(json_file)
```

View Data

We start of by viewing the data and calculating the RPM. This can be done with the following formula:

$$V_r = \dot{\theta} = \frac{d}{dt}(\theta) \cdot \frac{60}{360} \quad (1)$$

```

In [ ]: # Select example sample from data set
sample = 3
data_type = 0 # (Angle = 0, V_bus = 1, V_shunt = 2, current = 3)

# Create new dataframe for rotor data, containing rotor position and time
rotor_data = pd.concat([raw_df.iloc[:, (5*sample + data_type)], raw_df.iloc[:, (5*sample + data_type + 1)]], axis=1)
rotor_data.columns = ['rotor_position', 'Time']

# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Plotting
## Create Figure
plt.figure(figsize=(10, 6))

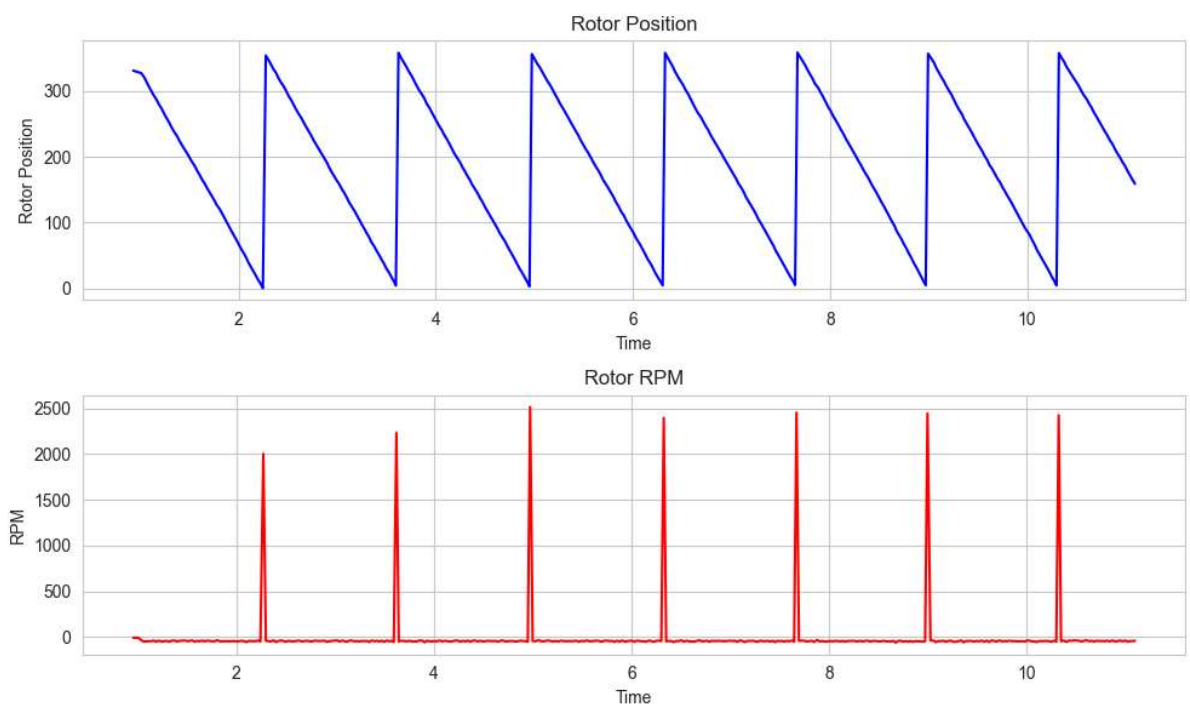
## Rotor position plot
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['rotor_position'], color='blue')
plt.title("Rotor Position")
plt.xlabel('Time')
plt.ylabel('Rotor Position')

## Motor RPM plot
plt.subplot(2, 1, 2)
plt.plot(rotor_data['Time'][1:], rotor_data['RPM'][1:], color='red') # Omit first
plt.title("Rotor RPM")
plt.xlabel('Time')
plt.ylabel('RPM')

# Save figure
file_name = "raw_rotor_pos_and_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

## Show plot
plt.tight_layout()
plt.show()

```



From the plot above, we see we counter two issues with the data:

- The rotor position goes from 0-359 degrees, resetting every 360 degrees.
- Every time the rotor position resets, the estimated velocity goes to the moon.
- The motor spins counter clock wise (CCW).

In order to improve the angular velocity estimate, we can do two things:

- Invert the rotor position data, from 359-0 to 0-359 degrees.
- Remove all negative values from the angular velocity estimate, since we know the motor spins in one direction at a constant speed.

```
In [ ]: # Invert data
rotor_data['rotor_position'] = 359 - rotor_data['rotor_position']

# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Remove negative values
rotor_data = rotor_data[rotor_data['RPM'] >= 0]

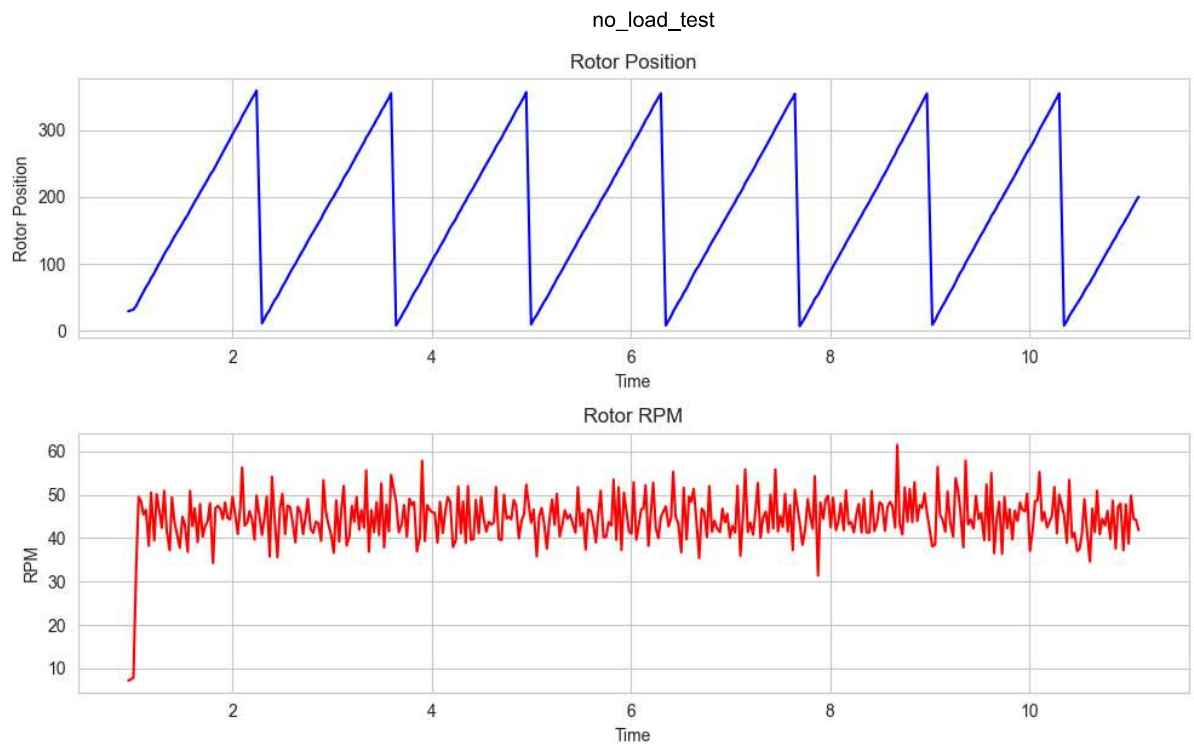
# Plotting
## Create Figure
plt.figure(figsize=(10, 6))

## Rotor position plot
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['rotor_position'], color='blue')
plt.title("Rotor Position")
plt.xlabel('Time')
plt.ylabel('Rotor Position')

## Motor RPM plot
plt.subplot(2, 1, 2)
plt.plot(rotor_data['Time'], rotor_data['RPM'], color='red')
plt.title("Rotor RPM")
plt.xlabel('Time')
plt.ylabel('RPM')
plt.tight_layout()

# Save figure
file_name = "corrected_rotor_pos_and_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

## Show plot
plt.show()
```



Now the data is looking a lot more reasonable. But we can still see that the speed data is containing some artifacts. We can filter this out with outlier detection, and interpolate the missing values.

```
In [ ]: # Calculate z-scores for each value in the RPM column
z_scores = stats.zscore(rotor_data['RPM'])

# Define a threshold for identifying outliers (e.g., 3 standard deviations from the
threshold = 3

# Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

# Remove outliers from the DataFrame
cleaned_rotor_data = rotor_data[~outlier_indices]

# Calculate mean RPM before removing outliers
mean_rpm_before = rotor_data['RPM'].mean()

# Calculate mean RPM after removing outliers
mean_rpm_after = cleaned_rotor_data['RPM'].mean()

# Plot 'RPM' column before removing outliers
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(rotor_data['Time'], rotor_data['RPM'], color='blue')
plt.axhline(y=mean_rpm_before, color='green', linestyle='--', label=f'Mean RPM: {mean_rpm_before}')
plt.title('Before Removing Outliers')
plt.xlabel('Time')
plt.ylabel('RPM')
plt.legend()

# Plot 'RPM' column after removing outliers
plt.subplot(2, 1, 2)
plt.plot(cleaned_rotor_data['Time'], cleaned_rotor_data['RPM'], color='red')
plt.axhline(y=mean_rpm_after, color='green', linestyle='--', label=f'Mean RPM: {mean_rpm_after}')
plt.title('After Removing Outliers')
plt.xlabel('Time')
plt.ylabel('RPM')
```

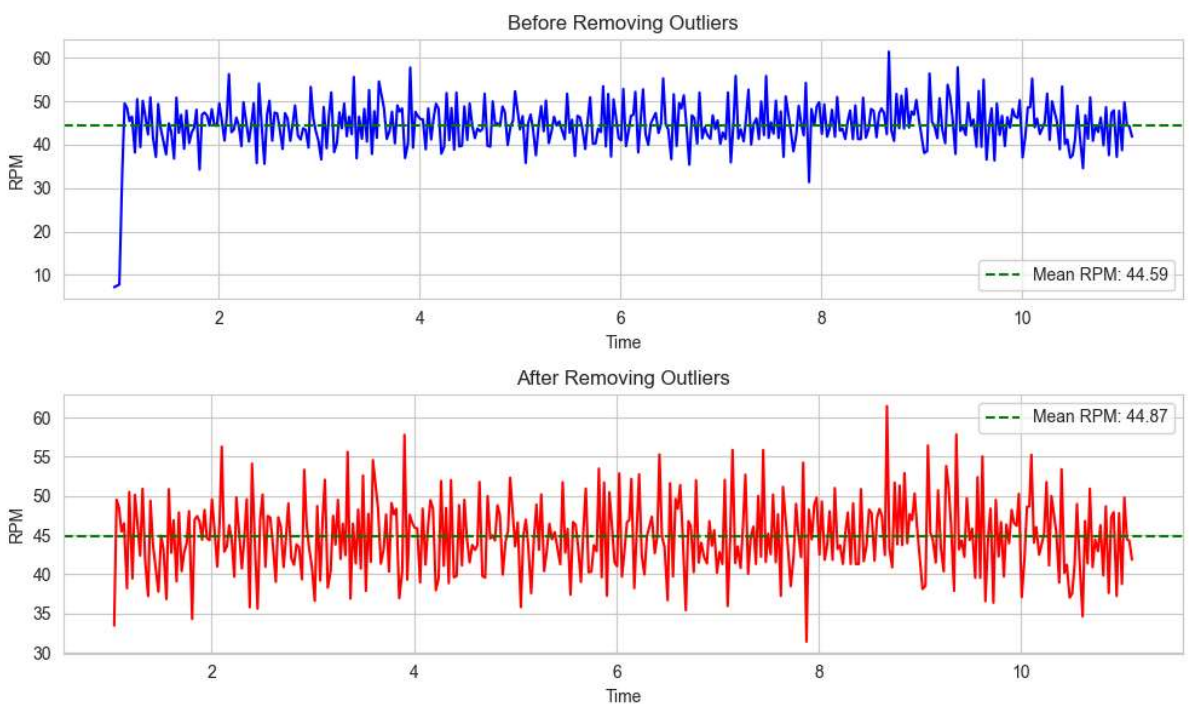
```
plt.legend()
plt.tight_layout()

# Save figure
file_name = "filtered_and_unfiltered_RPM.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show plot
plt.show()

# Update RPM in rotor_data
rotor_data['RPM'] = cleaned_rotor_data['RPM']

# Print mean RPM before and after outlier removal
print(f"The mean RPM before outlier removal is: {mean_rpm_before:.4f} RPM")
print(f"The mean RPM after outlier removal is: {mean_rpm_after:.4f} RPM")
print(f"This amounts to a difference of: {abs(mean_rpm_before - mean_rpm_after):.4f} RPM")
```



The mean RPM before outlier removal is: 44.5858 RPM

The mean RPM after outlier removal is: 44.8719 RPM

This amounts to a difference of: 0.2861 RPM

With this data filtering, we will assume that we have a decent data set, and can apply this to the entirety of the data.

```
In [ ]: RPM_df = pd.DataFrame()

# Find length of columns
num_columns = len(raw_df.columns)

# Iterating through every fifth column
for sample in range(0, num_columns, 5):
    # Create new dataframe for rotor data, containing rotor position and time
    rotor_data = pd.concat([raw_df.iloc[:, (sample)], raw_df.iloc[:, (sample + 4)]])
    rotor_data.columns = ['rotor_position', 'Time']

## Invert data
rotor_data['rotor_position'] = 359 - rotor_data['rotor_position']
```

```
# Calculate motor RPM from raw data
rotor_data['RPM'] = rotor_data['rotor_position'].diff() / rotor_data['Time'].diff()

# Remove negative values
rotor_data = rotor_data[rotor_data['RPM'] >= 0]

# Calculate z-scores for each value in the 'RPM' column
z_scores = stats.zscore(rotor_data['RPM'])

# Define a threshold for identifying outliers (e.g., 3 standard deviations from
threshold = 3

# Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

# Remove outliers from the DataFrame
cleaned_rotor_data = rotor_data[~outlier_indices]

# Add to RPM dataframe
RPM_df[f'{(sample/5) * voltage_step}V'] = cleaned_rotor_data['RPM']

# Final data clean up
# Replace "inf" values with NaN
RPM_df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Replace NaN values in each column with interpolation
RPM_df = RPM_df.interpolate()

# Remove first rows with NaN values.
nan_index = RPM_df.index[RPM_df.isnull().any(axis=1)][0]
RPM_df = RPM_df.iloc[nan_index + 2:]

# Remove non-zero values from first column, we know the speed of the motor was zero
RPM_df.iloc[:, :3] = 0

# Reset index to start from zero
RPM_df = RPM_df.reset_index(drop=True)

# Display Table
rounded_df = RPM_df.round(1)
rounded_df
```

Out[]:

	0.0V	0.5V	1.0V	1.5V	2.0V	2.5V	3.0V	3.5V	4.0V	4.5V	...	12.5V	13.0V	13.5V	14.0V
0	0.0	0.0	0.0	49.5	81.7	98.8	118.2	193.8	203.9	286.2	...	893.9	946.9	931.2	710.0
1	0.0	0.0	0.0	48.5	85.0	125.3	148.8	154.0	203.4	252.6	...	958.9	931.9	835.8	810.0
2	0.0	0.0	0.0	45.4	71.2	105.4	137.4	172.9	268.4	231.2	...	652.9	688.8	782.5	810.0
3	0.0	0.0	0.0	38.2	70.2	107.9	136.2	194.4	230.4	253.1	...	710.0	792.3	768.1	810.0
4	0.0	0.0	0.0	50.5	88.9	112.7	120.6	157.8	202.8	253.1	...	877.9	920.0	1056.4	910.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
343	0.0	0.0	0.0	47.9	98.1	97.9	152.9	150.3	204.6	244.6	...	818.3	698.9	745.3	1010.0
344	0.0	0.0	0.0	37.2	71.4	103.9	178.5	158.1	236.5	231.7	...	874.1	845.6	790.4	1010.0
345	0.0	0.0	0.0	47.8	73.2	133.0	162.2	165.9	195.7	218.8	...	687.1	817.0	835.6	1010.0
346	0.0	0.0	0.0	49.8	76.1	129.0	145.8	168.3	216.8	246.0	...	765.6	788.5	830.6	1110.0
347	0.0	0.0	0.0	44.5	70.1	118.9	136.7	193.8	228.0	249.7	...	775.9	788.5	825.6	1110.0

348 rows × 35 columns

Perform Linear Regression on the data set to test data

```
In [ ]: # Initialize empty numpy arrays for x and y
x = np.array([])
y = np.array([])

n = 0
# Loop through each column in the DataFrame
for column in RPM_df.columns:
    # Add values from the column to y array
    x = np.append(x, RPM_df[column].values)

    # Add an array of 2s to x array (assuming the same length as y)
    y = np.append(y, np.full_like(RPM_df[column].values, n*0.5))
    n += 1

# Perform Linear regression
slope, intercept, _, _, stderr = stats.linregress(x, y)
print("Regression line: y(x) = {:.3f} + {:.4f}x.".format(intercept, abs(slope)))
print("Regression line: V(RPM) = {:.3f}V + {:.4f}*RPM.".format(intercept, abs(slope)))
# Calculate confidence interval
conf_interval = 1.96 * stderr # 95% confidence interval

# Create scatter plot of x and y values with seaborn
sns.scatterplot(x=x, y=y)

# Plot the regression line with confidence interval using seaborn
sns.lineplot(x=x, y=slope * x + intercept, color='red', label='Linear Regression',

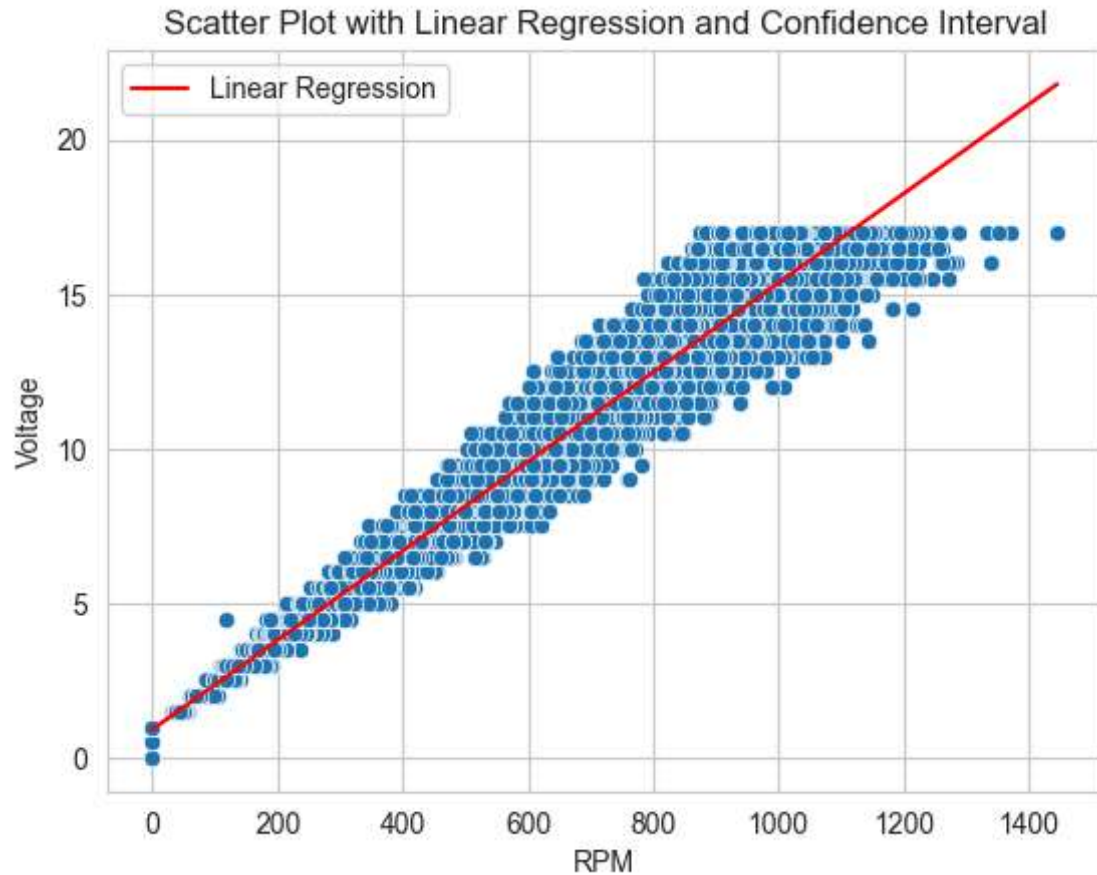
# Add labels and title
plt.xlabel('RPM')
plt.ylabel('Voltage')
plt.title('Scatter Plot with Linear Regression and Confidence Interval')
plt.legend() # Show Legend
```

```
# Save figure
file_name = "motor_constant_lin_reg.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show the plot
plt.show()
```

Regression line: $y(x) = 0.905 + 0.0145x$.

Regression line: $V(\text{RPM}) = 0.905V + 0.0145 \cdot \text{RPM}$.



We need to calculate the back-EMF E , in order to get a realistic estimate for the motor constant. We can do this with the following formula:

$$E = K_m \dot{\theta} = (V - i_a \cdot R_a - V_{shunt}) \quad (2)$$

```
In [ ]: # Create an empty DataFrame for motor constant
K_motor_df = pd.DataFrame()

# Iterating through every fifth column
for sample in range(0, len(raw_df.columns), 5):
    # Import data
    ## Create temporary dataframe for motor data
    temp_df = pd.concat([raw_df.iloc[:, (sample + 4)], raw_df.iloc[:, sample], raw_df.iloc[:, sample+1], raw_df.iloc[:, sample+2]], axis=1)
    temp_df.columns = ['Time', 'rotor_position', 'current', 'V_shunt']

    ## Rescale data
    temp_df['rotor_position'] = 359 - temp_df['rotor_position'] # Reverse rotation
    temp_df['current'] /= 1000 # Convert from mA to A
    temp_df['V_shunt'] /= 1000 # Convert from mV to V

    # Calculate motor RPM from raw data
    temp_df['RPM'] = temp_df['rotor_position'].diff() / temp_df['Time'].diff()
```

```

temp_df['RPM'] *= (60 / 360) # Convert from deg/s to RPM

# Calculate BEMF [E]
## Add bus voltage
temp_df['V'] = voltage_step * (sample / 5) * np.ones(len(raw_df))

## Calculate IR
temp_df['V_IR'] = (temp_df['current']) * motor_data['R_a'] # Calculate IR.

## Calculate back EMF (BEMF)(E)
temp_df['V_E'] = temp_df['V'] - temp_df['V_IR'] - (temp_df['V_shunt'])

# Clean up data
## Remove negative values
temp_df = temp_df[temp_df['RPM'] >= 0]

## Replace "inf" values with NaN
temp_df.replace([np.inf, -np.inf], np.nan, inplace=True)

## Remove values above 2000 RPM
temp_df = temp_df[temp_df['RPM'] <= 2000]

## Calculate z-scores for each value in the 'RPM' column
z_scores = stats.zscore(temp_df['RPM'])

## Define a threshold for identifying outliers (e.g., 3 standard deviations from
threshold = 3

## Find the indices of outliers
outlier_indices = np.abs(z_scores) > threshold

## Remove outliers from the DataFrame
temp_df = temp_df[~outlier_indices]

## Replace NaN values in each column with interpolation
temp_df = temp_df.interpolate()

# Append temp_df to K_motor_df
K_motor_df = pd.concat([K_motor_df, temp_df], ignore_index=True)

## Reset index to start from zero
K_motor_df = K_motor_df.reset_index(drop=True)

# Save data
if SAVE:
    file_name = "K_motor.csv" # CSV name
    file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
    K_motor_df.to_csv(file_path, index=False) # Save df

# Display Table
rounded_df = K_motor_df.round(1)
rounded_df

```

Out[]:

	Time	rotor_position	current	V_shunt	RPM	V	V_IR	V_E
0	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
1	1.0	212.5	-0.0	0.0	0.6	0.0	-0.0	0.0
2	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
3	1.0	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
4	1.1	212.4	0.0	0.0	0.0	0.0	0.0	-0.0
...	...	...	...	...	...	...	...	...
10602	11.5	321.5	0.2	0.0	1209.9	17.0	0.9	16.1
10603	11.5	305.1	0.2	0.0	1192.7	17.0	1.0	15.9
10604	11.6	294.8	0.2	0.0	1206.9	17.0	1.1	15.9
10605	11.6	278.6	0.1	0.0	1013.7	17.0	0.6	16.4
10606	11.7	262.3	0.2	0.0	1209.0	17.0	1.1	15.9

10607 rows × 8 columns

Now with this new data, we can redo the linear regression.

In []:

```

# Extract x and y values
x = K_motor_df['RPM'].values
y = K_motor_df['V_E'].values

# Perform linear regression
slope, intercept, _, _, stderr = stats.linregress(x, y)
print("Regression line: y(x) = {:.3f} + {:.4f}x.".format(intercept, abs(slope)))
print("Regression line: V(RPM) = {:.3f}V + {:.4f}*RPM.".format(intercept, abs(slope)))
print("Motor Constant: ({:.2f}mV / RPM ).".format(slope*1000))

# Calculate estimated values
K_motor_df['V_E_hat'] = slope * x + intercept

# Save data
if SAVE:
    file_name = "K_motor.csv" # CSV name
    file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
    K_motor_df.to_csv(file_path, index=False) # Save df

# Plot the regression line with confidence interval using seaborn
sns.lineplot(x=x, y=( K_motor_df['V_E_hat'].values ), color='red', label='Linear Re

# Create scatter plot of x and y values with seaborn
sns.scatterplot(x=x, y=y, label='Data points')

# Add Labels and title
plt.xlabel('RPM')
plt.ylabel('Voltage')
plt.title('Scatter Plot with Linear Regression')
plt.legend() # Show Legend

# Save figure
file_name = "motor_constant_w_o_IR.pdf" # Plot name
file_path = os.path.join(figure_folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

```

```

# Show the plot
plt.show()

# Save data
if SAVE:
    ## Generate file path
    file_path = os.path.join(json_folder_name, motor_file)

    ## Update K_V and K_M
    motor_data['K_V'] = slope
    motor_data['K_m'] = slope * ( (2*np.pi)/60)

    ## Save the updated dictionary to the JSON file
    with open(file_path, 'w') as json_file:
        json.dump(motor_data, json_file)

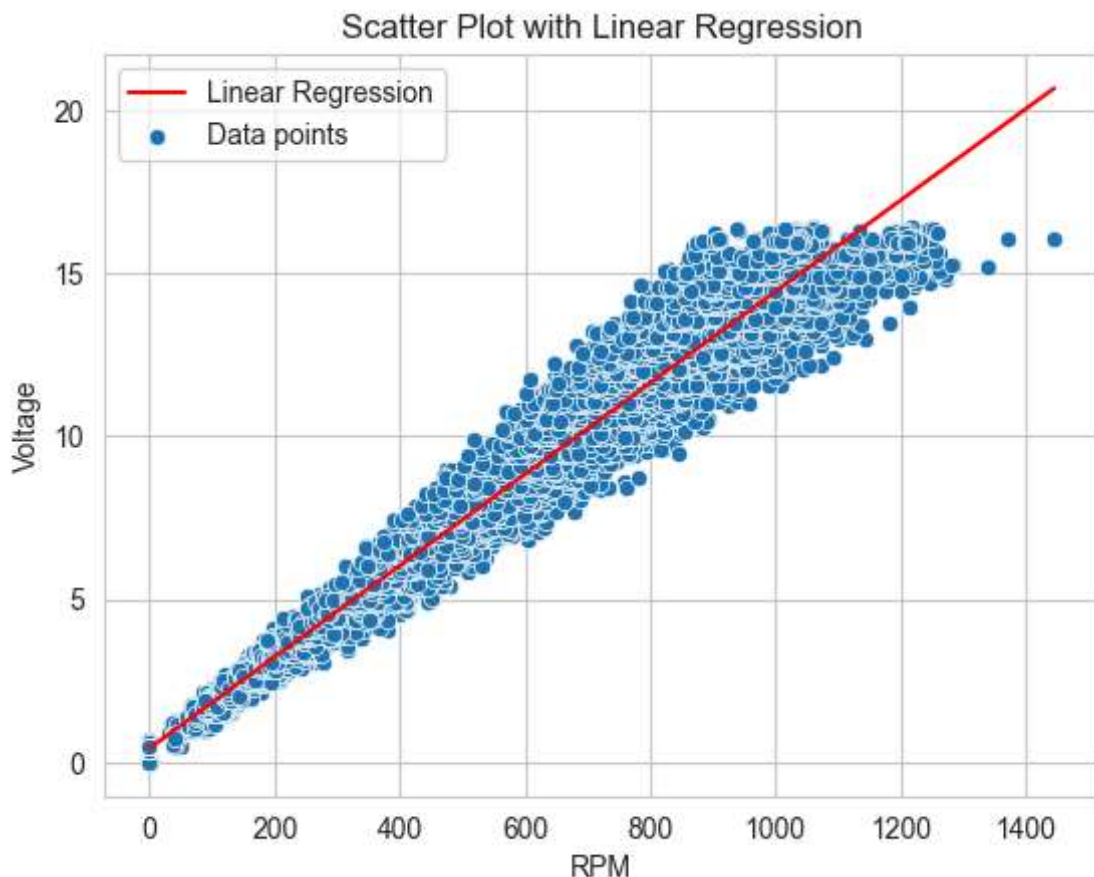
    ## Print new motor data
    print("New motor data:")
    print("R_a = {:.2f}".format(motor_data['R_a']))
    print("K_V = {:.4f}".format(motor_data['K_V']))
    print("K_m = {:.4f}".format(motor_data['K_m']))
    print("")

```

Regression line: $y(x) = 0.424 + 0.0140x$.

Regression line: $V(\text{RPM}) = 0.424\text{V} + 0.0140 \cdot \text{RPM}$.

Motor Constant: (14.01mV / RPM).



The data now clusters more evenly, which lines up with expectations. A comparison between the two is provided below:

```

In [ ]: # Initialize empty numpy arrays for x and y
x = np.array([])
y = np.array([])
x2 = K_motor_df['RPM'].values

```

```

y2 = K_motor_df['V_E'].values

n = 0
# Loop through each column in the DataFrame
for column in RPM_df.columns:
    # Add values from the column to y array
    x = np.append(x, RPM_df[column].values)

    # Add an array of 2s to x array
    y = np.append(y, np.full_like(RPM_df[column].values, n*0.5))
    n += 1

# Create a figure and two subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10))

# Plot the first subplot
sns.scatterplot(x=x, y=y, ax=ax1)
ax1.set_xlabel('RPM')
ax1.set_ylabel('Voltage')
ax1.set_title('Data points using bus voltage')
ax1.legend()

# Plot the second subplot
sns.scatterplot(x=x, y=y2, ax=ax2)
ax2.set_xlabel('RPM')
ax2.set_ylabel('Voltage')
ax2.set_title('Data points using back-EMF.')

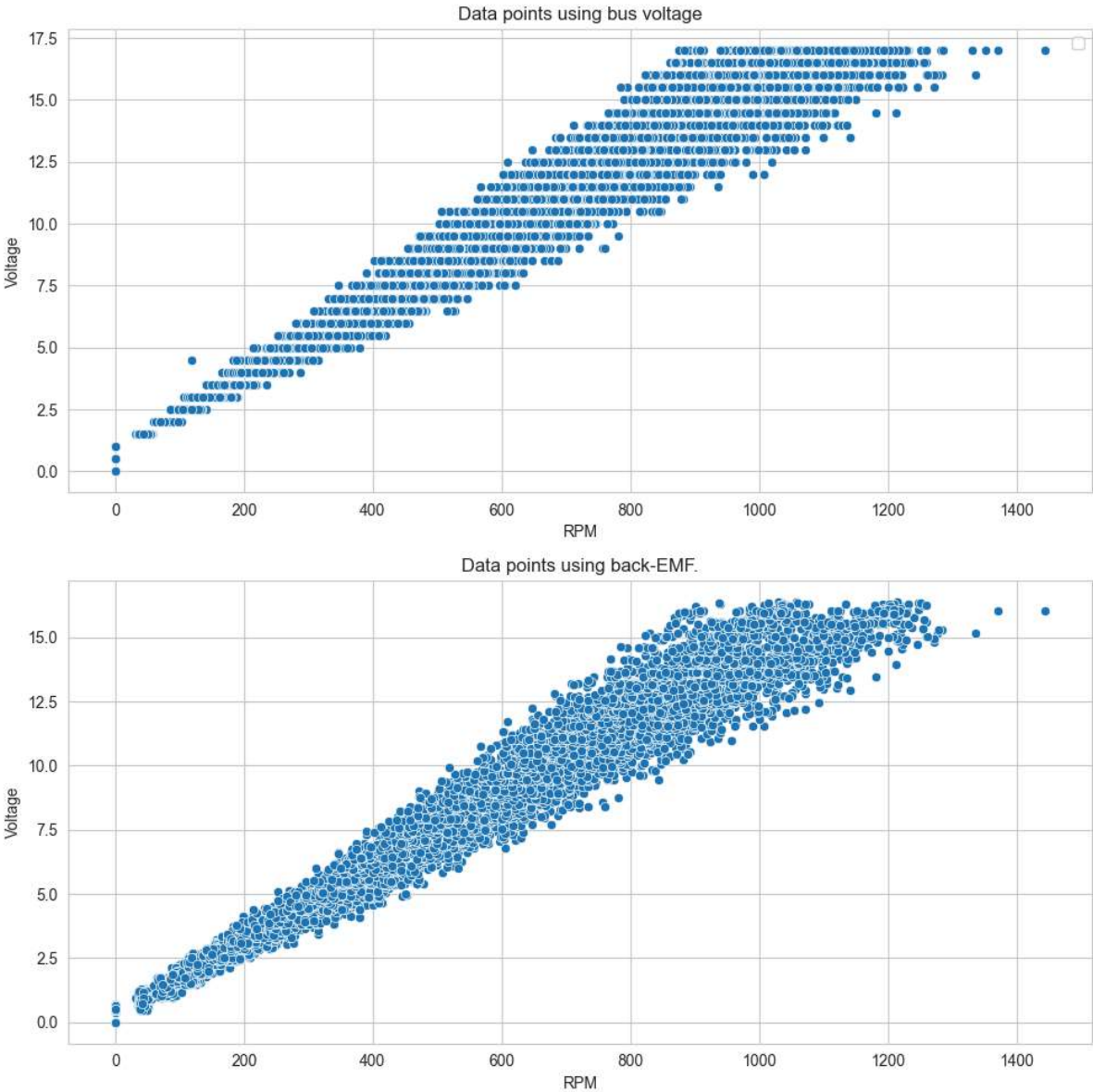
# Figure settings
plt.tight_layout()

# Save figure
file_name = "voltage_comparison.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Show the plot
plt.show()

```

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



Statistical Analysis and Inference

Packages

```
In [ ]: # Packages
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
import seaborn as sns
import os
import json
```

Configuration

```
In [ ]: # Folder name
CSV_folder_name = "CSV_files"
figure_folder_name = "figures"
json_folder_name = "json"

# Set Seaborn style
sns.set_style("whitegrid")

# Option to save data and figures ( True = Save data and figures, False = Don't save)
SAVE = False
```

Load Data Set

```
In [ ]: # Load data from CSV file
file_name = "K_motor.csv" # CSV name
file_path = os.path.join(CSV_folder_name, file_name) # Generate file path
motor_df = pd.read_csv(file_path) # Read CSV file.

# Load data from json file
motor_file = "motor_data.json" # CSV name
file_path = os.path.join(json_folder_name, motor_file) # Generate file path
with open(file_path, 'r') as json_file:
    motor_data = json.load(json_file)
```

Perform Regression Analysis in Deviation Form

$$\hat{y} = \alpha + \beta(x - \bar{x})$$

```
In [ ]: # Extract relevant data from dataframe
x = np.array(motor_df['RPM'].values) # Extract predictor values
y = np.array(motor_df['V_E'].values) # Extract target values

# Linear regression in deviation form
## Calculate predictor in deviation form
x_mean = np.mean(x)
x_dev = x - x_mean
```

```

## Perform linear regression using linregress
slope, intercept, r_value, p_value, std_err = stats.linregress(x_dev, y)

## Calculate predicted values and residuals
y_pred = intercept + slope * x_dev
residuals = y_pred - y

## Calculate SS_e, S_e^2, S_e and R^2
n = len(np.array(motor_df['V_E'].values))
SS_e = np.sum(residuals ** 2)
S_e_squared = SS_e / (n - 2)
S_e = np.sqrt(S_e_squared)
r_squared = r_value**2

## Print Results
print(" Linear Regression Analysis")
print("=====")
print("- Regression line:  $y(x) = {:.2f} + {:.3f}x$ ".format(intercept, abs(slope)))
print("-  $a = {:.4f}$ ".format(intercept))
print("-  $b = {:.4f}$ ".format(slope))
print("-  $SS_e = {:.4f}$ ".format(SS_e))
print("-  $S_e^2 = {:.4f}$ ".format(S_e_squared))
print("-  $S_e = {:.4f}$ ".format(S_e))
print("-  $R^2 = {:.2f}$ ".format(r_squared))

# Create Figure
## Plot datapoints
sns.scatterplot(x=x, y=y, color='black', label="Data points")

## Plot regression line
sns.lineplot(x=x, y=y_pred, color='red', linewidth=2, label="Regression line")

## Plot settings
plt.xlabel('Angular velocity [RPM]')
plt.ylabel('Back EMF [V]')
plt.title('Relationship Between Angular Velocity and Back-EMF.')
plt.xlim(0, 1350)
plt.ylim(0, 17)
plt.yticks(range(0, 18, 1))
plt.legend()

# Save figure
file_name = "lin_reg_RPM_BEMF.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

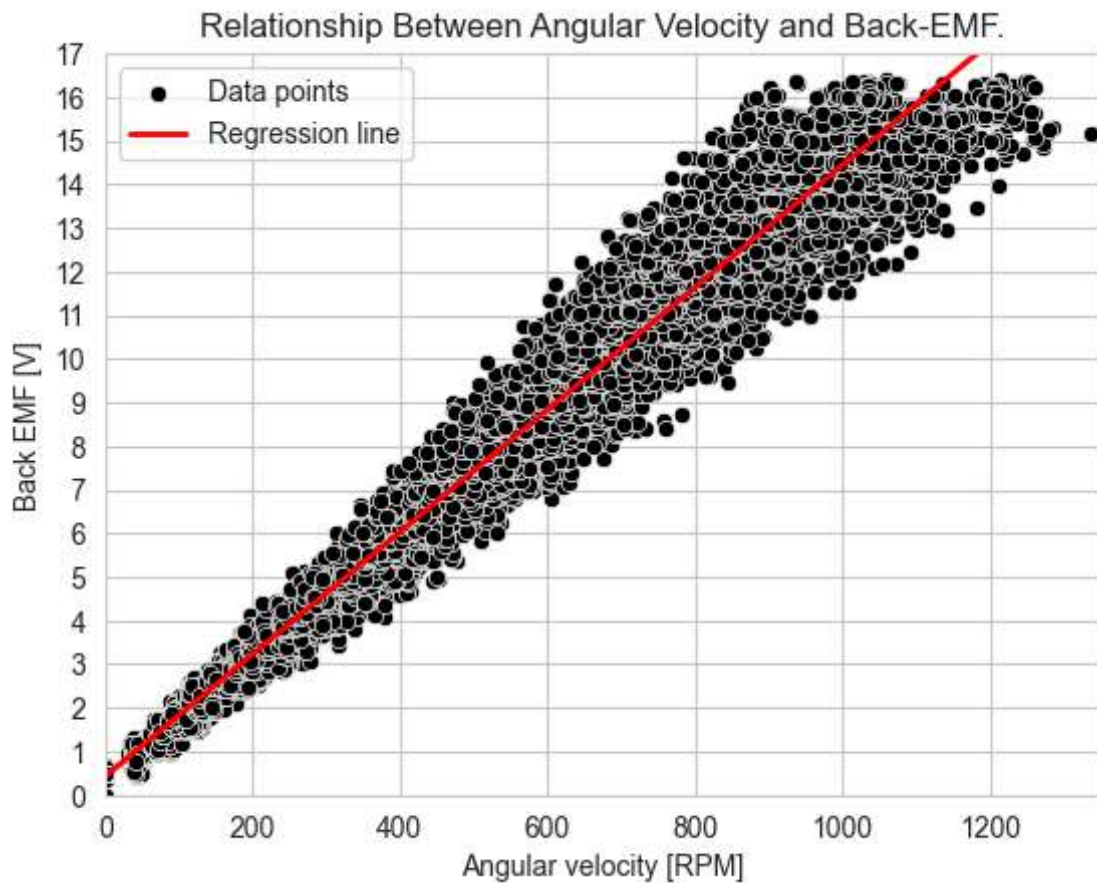
# Show plot
plt.show()

```

```

Linear Regression Analysis
=====
- Regression line:  $y(x) = 6.92 + 0.014x$ 
-  $a = 6.9184$ 
-  $b = 0.0140$ 
-  $SS_e = 6944.4464$ 
-  $S_e^2 = 0.6548$ 
-  $S_e = 0.8092$ 
-  $R^2 = 0.97$ 

```



Perform Inference

Hyper Parameters

```
In [ ]: # Prior Hyperparameters
sigma_0 = 0
n_0 = 0

## Priors P0
v_0 = n_0 - 2
S_0 = sigma_0**2 * max(0, v_0)

## Print prior
print("Prior hyperparameters:")
print("- v_0 = {:.2f}".format(v_0))
print("- S_0 = {:.2f}".format(S_0))
print("")

# Posterior P1
## Calculate posterior
v_1 = v_0 + n
S_1 = S_0 + SS_e
if n_0 == 0 & S_0 == 0:
    sigma_1 = np.sqrt( SS_e / (n - 2) )
else:
    sigma_1 = np.sqrt( S_1 / v_1 )

## Print residuals
print("Posterior hyperparameters:")
print("- x_mean = {:.4f}".format(x_mean))
print("- beta_vec = [{:.4f}, {:.4f}].format(intercept, slope))
print("- v_1 = {:.4f}".format(v_1))
```

```
print("- S_1 = {:.4f}".format(S_1))
print("- sigma_1 = {:.4f}".format(sigma_1))
print("- n = {:.4f}".format(n))
```

Prior hyperparameters:

```
- v_0 = -2.00
- S_0 = 0.00
```

Posterior hyperparameters:

```
- x_mean = 463.4078
- beta_vec = [6.9184, 0.0140]
- v_1 = 10605.0000
- S_1 = 6944.4464
- sigma_1 = 0.8092
- n = 10607.0000
```

Posterior Probability Distributions

Print distributions

```
In [ ]: # Calculate missing parameters
SS_x = np.sum(x_dev**2)

# Print parameters
print("Parameters:")
print("v_1/2 = {:.4f}".format(v_1/2))
print("S_1 = {:.4f}".format(S_1/2))
print("alpha_* = {:.4f}".format(intercept))
print("sigma_1 * sqrt(1/n) = {:.4f}*10^3".format(sigma_1*np.sqrt(1/n) * 10**3))
print("beta = {:.4f}".format(intercept))
print("sigma_1 * sqrt(1/SS_x) = {:.4f}*10^5".format(sigma_1*np.sqrt(1/SS_x) * 10**5))
print("x_mean = {:.4f}".format(x_mean))
print("1/n = {:.4f}*10^5".format((1/n) * 10**5))
print("1/SS_x = {:.4f}*10^10".format((1/SS_x) * 10**10))
print("")

# Print distributions
print("Distributions:")
print("tau ~ ({:.4f}, {:.4f})".format(v_1/2, S_1/2))
print("alpha_* ~ ({:.4f}, {:.4f}*10^3, {:.0f})".format(intercept, (sigma_1*np.sqrt(1/n) * 10**3), 0))
print("beta ~ ({:.4f}, {:.4f}*10^5, {:.0f})".format(slope, (sigma_1*np.sqrt(1/SS_x) * 10**5), 0))
print("y(x) ~ ({:.4f} + {:.4f}(x-{: .4f}), {:.4f} * sqrt[ {:.4f}*10^5) + ({:.4f}*10^10) )".format(
    intercept, slope, x_mean, sigma_1, ((1/n) * 10**5), ((1/SS_x) * 10**10)))
print("Y_+(x) ~ ({:.4f} + {:.4f}(x-{: .4f}), {:.4f} * sqrt[ 1 + ({:.4f}*10^5) + ({:.4f}*10^10) )".format(
    intercept, slope, x_mean, sigma_1, ((1/n) * 10**5), ((1/SS_x) * 10**10)))
```

Parameters:

$v_{1/2} = 5302.5000$

$S_1 = 3472.2232$

$\alpha_* = 6.9184$

$\sigma_1 * \sqrt{1/n} = 7.8572 \cdot 10^3$

$\beta = 6.9184$

$\sigma_1 * \sqrt{1/SS_x} = 2.3377 \cdot 10^5$

$x_{\text{mean}} = 463.4078$

$1/n = 9.4277 \cdot 10^5$

$1/SS_x = 8.3453 \cdot 10^{-10}$

Distributions:

$\tau \sim (5302.5000, 3472.2232)$

$\alpha_* \sim (6.9184, 7.8572 \cdot 10^3, 10605)$

$\beta \sim (0.0140, 0.0234 \cdot 10^5, 10605)$

$y(x) \sim (6.9184 + 0.0140(x-463.4078), 0.8092 * \sqrt{(9.4277 \cdot 10^5) + (8.3453 \cdot 10^{-10})(x-463.4078)^2}), 10605)$

$Y_+(x) \sim (6.9184 + 0.0140(x-463.4078), 0.8092 * \sqrt{1 + (9.4277 \cdot 10^5) + (8.3453 \cdot 10^{-10})(x-463.4078)^2}), 10605)$

Plot Distributions

```
In [ ]: # Generate Values
## Generate x_values
x1_points = np.linspace(1.25, 1.75, 1000)
x2_points = np.linspace(intercept - 4*sigma_1*np.sqrt(1/n), intercept + 4*sigma_1*np.sqrt(1/n), 1000)
x3_points = np.linspace(slope - 4*sigma_1*np.sqrt(1/SS_x), slope + 4*sigma_1*np.sqrt(1/SS_x), 1000)
x4_points = np.linspace(x_mean - 4*np.std(x, ddof=1) / np.sqrt(n), x_mean + 4*np.std(x, ddof=1) / np.sqrt(n), 1000)

## Precision
tau_gamma = stats.gamma.pdf(x1_points, v_1/2, scale=1/(S_1/2))

## Intercept
alpha_star_t = stats.t.pdf(x2_points, v_1, loc=intercept, scale=sigma_1*np.sqrt(1/n))

## Slope
beta_t = stats.t.pdf(x3_points, v_1, loc=slope, scale=sigma_1*np.sqrt(1/SS_x))

## Mean
mean_t = stats.t.pdf(x4_points, v_1, loc=x_mean, scale=np.std(x, ddof=1) / np.sqrt(n))

# Plotting
## Create figure
fig, axs = plt.subplots(2, 2, figsize=(6.3, 6.3/1.41))

## Precision subplot
axs[0, 0].plot(x1_points, tau_gamma, label='Precision (Gamma)', color='blue')
axs[0, 0].set_title('Precision (Gamma)')

## Intercept subplot
axs[1, 0].plot(x2_points, alpha_star_t, label='Intercept (t)', color='orange')
axs[1, 0].set_title('Intercept (t)')

## Slope subplot
axs[0, 1].plot(x3_points, beta_t, label='Slope (t)', color='green')
axs[0, 1].set_title('Slope (t)')

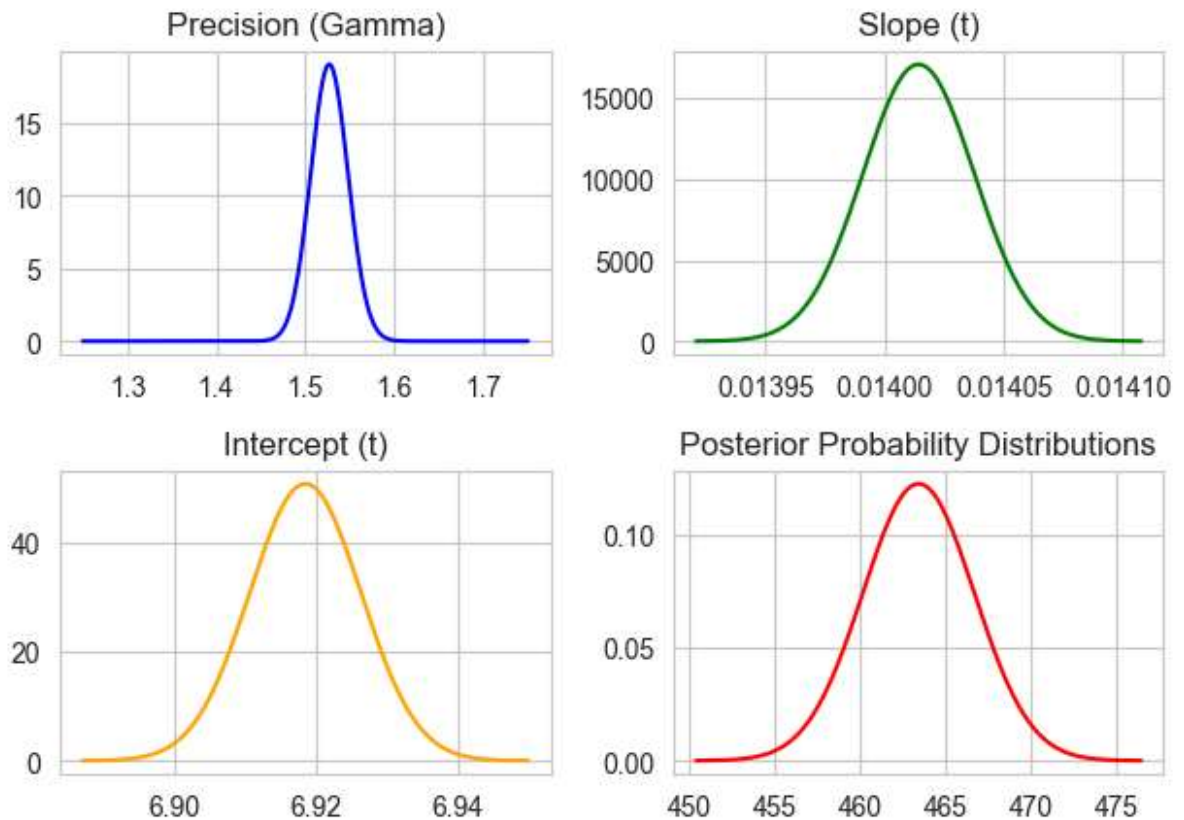
## Mean subplot
axs[1, 1].plot(x4_points, mean_t, label='Mean (t)', color='red')
axs[1, 1].set_title('Mean (t)')

# Figure settings
```

```
plt.title("Posterior Probability Distributions")
plt.grid(True)
plt.tight_layout()

# Save figure
file_name = "distributions.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    plt.savefig(file_path, format='pdf')

# Display Figure
plt.show()
```



Plotting Credibility and Predictive interval with regression line at 99% confidence.

```
In [ ]: # Confidence Level
P = 0.99
t_value = stats.t.ppf(P, n - 2)

# Calculate Margin of Error for credibility and prediction intervals
MoE_cred = t_value * sigma_1 * np.sqrt((1/n) + (1/SS_x)*(x-x_mean)**2)
MoE_pred = t_value * sigma_1 * np.sqrt(1 + (1/n) + (1/SS_x)*(x-x_mean)**2)

# Create Figure for prediction interval
figure_width_inches = 8.27
fig1, ax1 = plt.subplots(figsize=(figure_width_inches, figure_width_inches/1.41))
interval_fill_opacity = 0.10
interval_line_opacity = 0.25
fig1.suptitle("Prediction Interval", fontsize=16)

# Plot prediction interval
```

```

## Data points and regression line
sns.scatterplot(x=x, y=y, color='black', ax=ax1, label='Data Points')
sns.lineplot(x=x, y=y_pred, color='red', ax=ax1, label='Linear Regression')

## Intervals
ax1.fill_between(x, (y_pred-MoE_pred), (y_pred+MoE_pred), color='blue', alpha=inter
sns.lineplot(x=x, y=(y_pred-MoE_pred), color='blue', alpha=interval_line_opacity, a
sns.lineplot(x=x, y=(y_pred+MoE_pred), color='blue', alpha=interval_line_opacity, a

## Subplot settings
ax1.set(xlabel="Angular Velocity [RPM]", ylabel="Back EMF [V]")
ax1.legend(loc='upper left')
ax1.set_yticks(np.arange(0, 18, 1))
ax1.set_xlim(0, 1200)
ax1.set_ylim(0, 17)

# Save figure
file_name = "prediction_interval.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    fig1.savefig(file_path, format='pdf')

# Display figure
plt.show()

# Create Figure for credibility interval
fig2, ax2 = plt.subplots(figsize=(figure_width_inches, figure_width_inches/1.41))
fig2.suptitle("Credibility Interval", fontsize=16)

# Plot credibility interval
## Data points and regression line
sns.scatterplot(x=x, y=y, color='black', ax=ax2, label='Data Points')
sns.lineplot(x=x, y=y_pred, color='red', ax=ax2, label='Linear Regression')

## Intervals
ax2.fill_between(x, (y_pred-MoE_cred), (y_pred+MoE_cred), color='green', alpha=inter
sns.lineplot(x=x, y=(y_pred-MoE_cred), color='green', alpha=interval_line_opacity,
sns.lineplot(x=x, y=(y_pred+MoE_cred), color='green', alpha=interval_line_opacity,

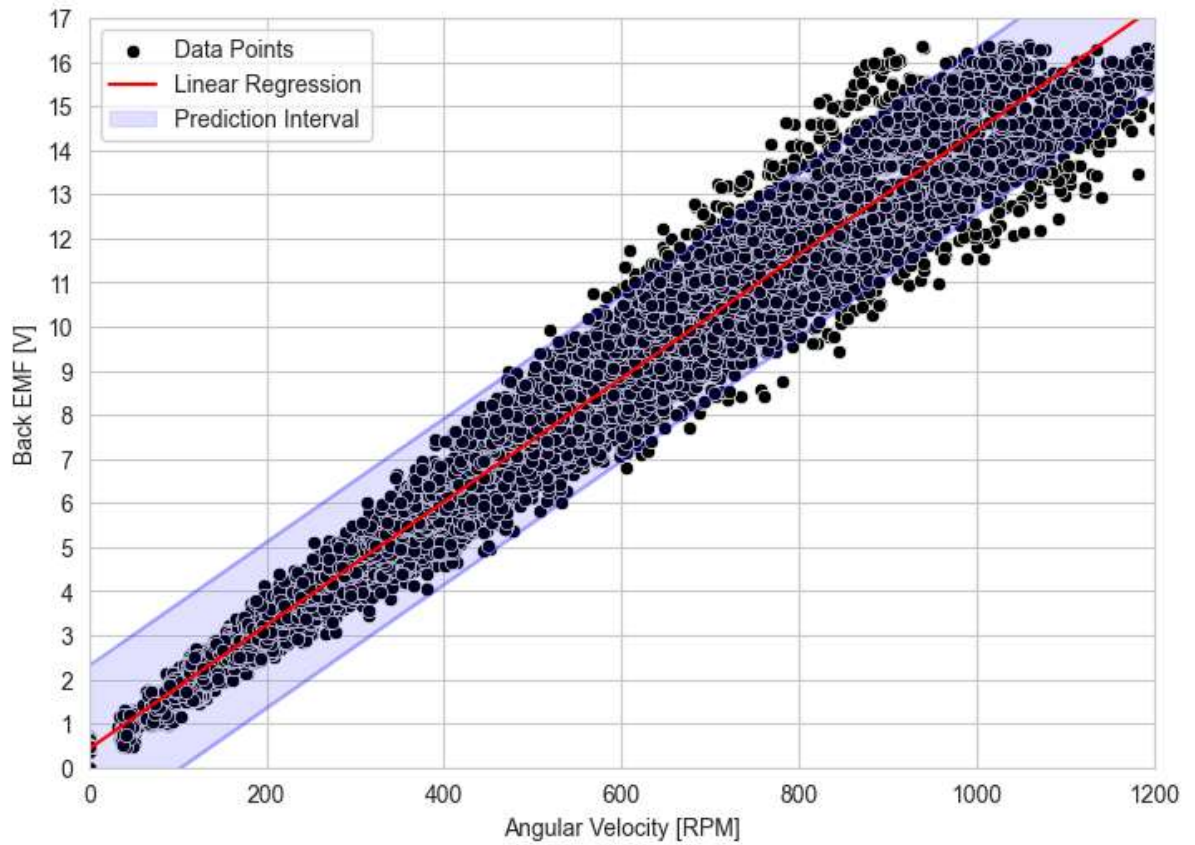
## Subplot settings
ax2.set(xlabel="Angular Velocity [RPM]", ylabel="Back EMF [V]")
ax2.legend(loc='upper left')
ax2.set_yticks(np.arange(0, 18, 0.25))
ax2.set_xlim(550, 600)
ax2.set_ylim(8, 9)

# Save figure
file_name = "credibility_interval.pdf" # Plot name
file_path = os.path.join(folder_name, file_name) # Generate file path
if SAVE:
    fig2.savefig(file_path, format='pdf')

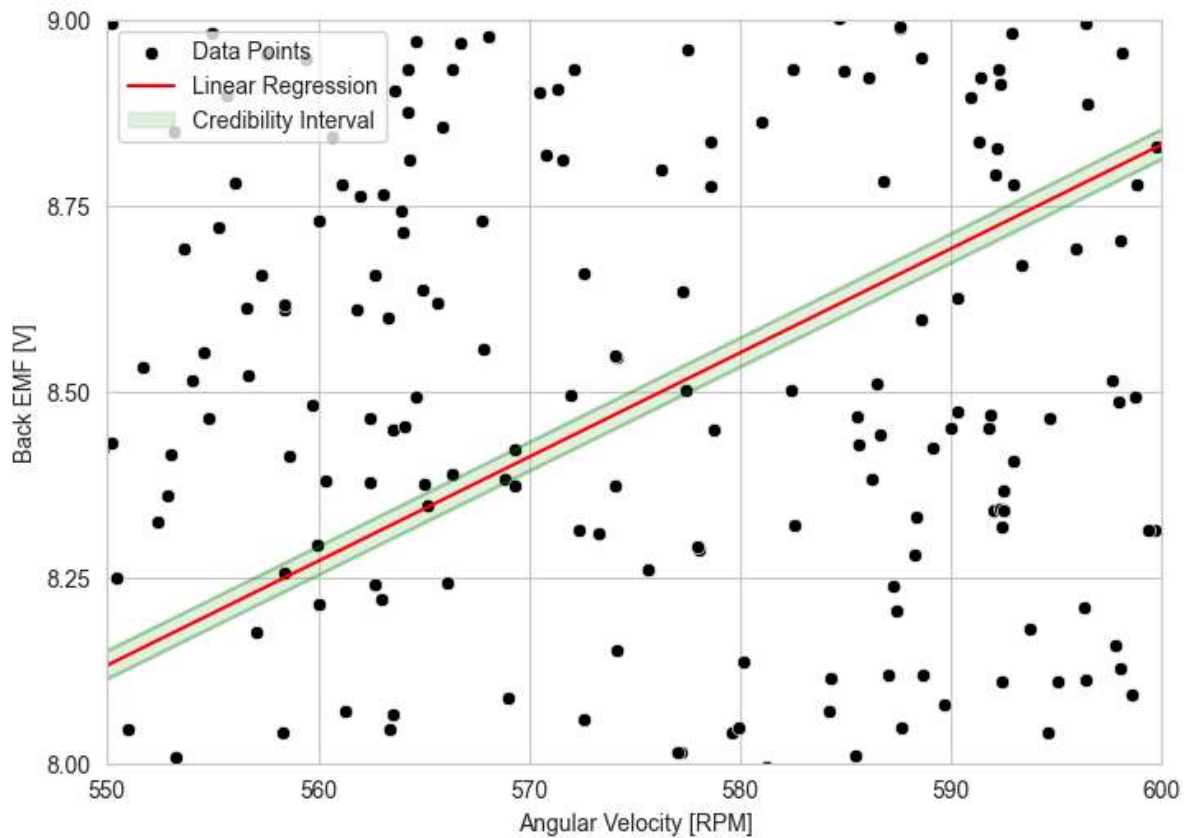
# Display figure
plt.show()

```

Prediction Interval



Credibility Interval



Calculate credibility and predictive interval for 800 RPM at 99% confidence.

```
In [ ]: # Calculate intervals at 800 RPM at 99% Confidence
# Calculate MoE
MoE_cred_800 = t_value * sigma_1 * np.sqrt((1/n) + (1/SS_x)*(800-x_mean)**2)
MoE_pred_800 = t_value * sigma_1 * np.sqrt(1 + (1/n) + (1/SS_x)*(800-x_mean)**2)

# Calculate intervals
cred_int_800 = [ (intercept + slope*(800-x_mean)) - MoE_cred_800, (intercept + slope*(800-x_mean)) + MoE_cred_800 ]
pred_int_800 = [ (intercept + slope*(800-x_mean)) - MoE_pred_800, (intercept + slope*(800-x_mean)) + MoE_pred_800 ]

# Print results
print("Credibility and Predictive interval at 800 RPM with 0.99 confidence.")
print("I_0.01 = ({:.4f}, {:.4f})".format(cred_int_800[0], cred_int_800[1]))
print("I_0.01^+ = ({:.4f}, {:.4f})".format(pred_int_800[0], pred_int_800[1]))
```

Credibility and Predictive interval at 800 RPM with 0.99 confidence.
I_0.01 = (11.6097, 11.6614)
I_0.01^+ = (9.7526, 13.5185)

Bibliografi

- [1] AMS. *AS5600*. URL: https://www.mouser.com/datasheet/2/588/AS5600_UG000254_2_00-1877354.pdf. (accessed: 07.05.2024).
- [2] Arduino. *Arduino® MEGA 2560 Rev3*. URL: <https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf>. (accessed: 07.05.2024).
- [3] Diligent. *Motor / Gearbox Resource Center*. URL: https://diligent.com/reference/motor_gearbox/motor_gearbox. (accessed: 07.05.2024).
- [4] EA ELEKTRO-AUTOMATIK. *EA-PS 2000 B TFT SINGLE 100 W - 320 W*. URL: <https://www.farnell.com/datasheets/3627616.pdf>. (accessed: 07.05.2024).
- [5] HALJIA. *HALJIA AS5600 Magnetic Encoder Induction Angle Measurement Sensor Module Magnetized Encoder 12 Bit High Precision with Magnet*. URL: <https://www.amazon.de/-/en/Magnetic-Induction-Measurement-Magnetized-Precision/dp/B08BCB899Q>. (accessed: 07.05.2024).
- [6] Texas Instruments. *INA219 Zero-Drift, Bidirectional Current/Power Monitor With I²C Interface*. URL: <https://www.ti.com/lit/ds/symlink/ina219.pdf>. (accessed: 07.05.2024).
- [7] reichelt. *DEBO SENS POWER Developer boards - current sensor with breakout board, INA219*. URL: <https://www.reichelt.com/ie/en/developer-boards-current-sensor-with-breakout-board-ina219-debo-sens-power-p266047.html>. (accessed: 07.05.2024).